

Virtualization Lab Guide

One hands-on lab per chapter, all on one 16 GB laptop running VMware Workstation and nested ESXi, from checking your CPU flags to running Kubernetes and breaking it on purpose.

How to use this guide

Each lab matches the hands-on exercise at the end of its chapter. Watch the lesson first; the lab assumes you know the concept and focuses on doing, observing, and explaining. Labs build on each other. The **Builds on** field tells you what must already exist, and the **RAM** field tells you what must be powered on.

Before Lab 3, complete Appendix A. It prepares your laptop, lists every download, and sets up the virtual networks the labs expect.

A few labs need more than one laptop can hold (vCenter, vMotion, vSphere HA). Those labs do everything possible locally, then finish in VMware Hands-on Labs, which are free and run in a browser.

How each lab is laid out

esxi# command

A command to type. Lines starting with # inside a block are comments.

EXPECTED OUTPUT
...

What you should see. Your values (addresses, IDs, timings, versions) will differ. Look for the shape of the output, not the exact numbers.

CHECKPOINT

Stop and confirm before moving on. If a checkpoint fails, fix it now. Later steps depend on it.

Expected results and notes for instructors or study partners for every lab are in **Appendix B**. Try each lab first, then check your work against it.

Prompt conventions

Prompt	Where you type it
PS>	PowerShell on your laptop (elevated where noted), or on a Windows Server VM in Lab 19
esxi# esxi1# esxi2#	SSH session to a nested ESXi host as root (or your named admin after Lab 27)
guest\$	Normal user inside a Linux guest VM (use sudo where shown)
stor1\$ pve1# cp\$ w\$	Storage server VM, a Proxmox node, a Kubernetes control-plane node, a Kubernetes worker
<value>	Replace with your own value, such as a VM ID or thumbprint

Difficulty



Foundation. Follow the steps; little can go wrong.



Intermediate. Expect to troubleshoot at least one step.



Advanced. Multi-host, deliberately destructive, or both. Snapshot first.

Lab index

Lab	Exercise	Time	Level	RAM
PART I • FOUNDATIONS				
01	Find the virtualization already in your life What Virtualization Actually Is	45 min	■□□	None
02	Confirm your CPU can virtualize How Virtualization Works Under the Hood	30 min	■□□	None
03	Stand up Workstation and a nested ESXi host Building Your Virtualization Lab	2–3 hrs	■■■	8 GB
04	Build a VM deliberately, setting by setting Your First Virtual Machine	90 min	■□□	2 GB
05	Write a diagnostic plan before touching anything The Virtualization Troubleshooting Mindset	45 min	■□□	None
PART II • VIRTUALIZING THE FOUR RESOURCES				
06	Overcommit CPU and watch ready time appear CPU Virtualization	90 min	■■■	5 GB
07	Drive a host from ballooning into swap Memory Virtualization	90 min	■■■	5 GB
08	Fill a thin-provisioned datastore on purpose Virtual Storage and Disks	90 min	■■■	5 GB
09	Connect two hosts to shared storage Shared Storage and the Datastore	2 hrs	■■■	11 GB
10	Wire VMs in bridged, NAT, and host-only modes Virtual Networking Fundamentals	75 min	■□□	4 GB
11	Separate traffic types across two hosts Advanced Virtual Networking	2 hrs	■■■	11 GB
PART III • OPERATING THE PLATFORM				
12	Snapshot chains, a golden template, and custom clones Snapshots, Clones, and Templates	2 hrs	■■■	4 GB
13	Shares, limits, reservations, and finding the bottleneck Resource Management and Performance	2 hrs	■■■	5 GB
14	Migrate cold locally, live in the Hands-on Lab, then break it Migration and Moving Running Workloads	2 hrs	■■■	11 GB

Lab	Exercise	Time	Level	RAM
15	Pull the plug on a host Clustering and High Availability	2 hrs	■■■	11 GB
16	Back up, delete, restore, and prove it boots Backup, Recovery, and Business Continuity	90 min	■■□	5 GB
17	Provision a fleet from a definition file Automation and Infrastructure as Code	2–3 hrs	■■■	5 GB
PART IV · THE MAJOR PLATFORMS				
18	Map every concept onto vSphere VMware vSphere	3 hrs	■■□	5 GB
19	Hyper-V VMs, a switch, and replica to a second host Microsoft Hyper-V	3 hrs	■■■	9 GB
20	A two-node Proxmox cluster with live migration Open Source Virtualization: KVM and Proxmox	2 hrs	■■□	7 GB
21	Deliver a virtual desktop through a broker Desktop and Application Virtualization	2 hrs	■■□	6 GB
PART V · CONTAINERS AND THE CLOUD				
22	First containers, measured against a VM From Virtual Machines to Containers	60 min	■□□	3 GB
23	Build, persist, and publish an image Docker and Container Fundamentals	90 min	■■□	3 GB
24	A small cluster that survives a node failure Container Orchestration and Kubernetes	2–3 hrs	■■■	5 GB
25	Trace one workload through every layer Containers and Virtual Machines Together	90 min	■■■	5 GB
26	A cloud VM, a managed container, and the map back Cloud Virtualization and the Bigger Picture	90 min	■■□	None local
PART VI · SECURITY, TROUBLESHOOTING, AND SCALE				
27	Harden the management plane and scan an image Virtualization Security	2 hrs	■■□	8 GB
28	Diagnose a multi-layer fault to root cause The Virtualization Troubleshooting Framework in Full	2 hrs	■■■	11 GB
29	Attribute four staged slowdowns Diagnosing Performance Problems	2 hrs	■■■	5 GB

Lab	Exercise	Time	Level	RAM
30	Recover from five staged failures Diagnosing Failures and Outages	3 hrs	■■■	11 GB
31	Size the host and reclaim the waste Capacity Planning and Running It at Scale	90 min	■□□	5 GB
32	Assess yourself and write a 90-day plan Building Skills and Where to Go Next	60 min	■□□	None

Foundations

Concepts, the lab itself, your first VM, and the troubleshooting method every later lab leans on.

LAB 01

Chapter 1 · What Virtualization Actually Is

Find the virtualization already in your life

Build an inventory of every place virtualization already runs for you, and classify each entry by type.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
45 min	Foundation	None		None
PLATFORM	Your laptop + a notebook			

You will

- Detect whether your own OS is running on a hypervisor
- Name the virtualization type behind five everyday services
- Explain for one example what was abstracted and what it bought

Before you start

- Admin access to your laptop
- A list of the cloud services you use weekly

Procedure

- 1
- Ask Windows whether it sees a hypervisor underneath it. Look at the last lines of the output.

```
PS> systeminfo | Select-String "Hyper-V|hypervisor"
```

EXPECTED OUTPUT

Hyper-V Requirements: A hypervisor has been detected. Features required for Hyper-V will not be displayed.

- 2
- Check whether Virtualization-Based Security is running. This is Windows running its own kernel as a guest of Hyper-V.

```
PS> (Get-CimInstance -Namespace root\Microsoft\Windows\DeviceGuard -ClassName Win32_DeviceGuard).VirtualizationBasedSecurityStatus
```

EXPECTED OUTPUT

2 # 2 = running, 1 = enabled but not running, 0 = off

- 3 On macOS or Linux, run the equivalent. 'none' means bare metal.

```
$ sysctl kern.hv_support            # macOS: 1 = available
$ systemd-detect-virt            # Linux: none, kvm, vmware, microsoft...
```

- 4 List every tool on your laptop that runs a virtual environment: WSL, Docker Desktop, an Android emulator, VMware Workstation, a browser sandbox.

CHECKPOINT Your inventory has at least three local entries.

- 5 List five services you used this week (email, streaming, a code host, banking, file sharing). For each, write the virtualization type most likely underneath: server, desktop, storage, network, or application.
- 6 Pick one entry and write three sentences: what physical thing was separated from the software, what that separation made possible, and what it costs.

Check your work

- ☐ Your OS returned a definite answer about a hypervisor
- ☐ Inventory has at least eight rows, each with a type
- ☐ One written example names a benefit and a cost

If it goes wrong

systeminfo says nothing about Hyper-V

Run it from an elevated prompt, or open msinfo32 and read 'Virtualization-based security'.

VBS status is 2

Note it now. Appendix A has you turn it off before Lab 3, because it blocks nested virtualization in Workstation.

CLEAN UP Nothing to clean up. Keep the inventory; you will revisit it in Lab 26.

Confirm your CPU can virtualize

Prove your laptop exposes hardware-assisted virtualization and second-level address translation, and that firmware has them switched on.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
30 min	 Foundation	None		Lab 1
PLATFORM	Your laptop			

You will

- Identify Intel VT-x or AMD-V
- Confirm EPT (Intel) or RVI/NPT (AMD)
- Know where firmware and Windows can silently hide them

Before you start

- Admin access
- Know how to enter your laptop's firmware setup (usually F2, F10, or Del at boot)

Procedure

- 1 Read the processor's virtualization properties.

```
PS> Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors, VirtualizationFirmwareEnabled, VMMonitorModeExtensions,
SecondLevelAddressTranslationExtensions
```

EXPECTED OUTPUT

```
Name                                : 13th Gen Intel(R) Core(TM) i7-1360P
NumberOfCores                       : 12
NumberOfLogicalProcessors           : 16
VirtualizationFirmwareEnabled       : True
VMMonitorModeExtensions             : True
SecondLevelAddressTranslationExtensions: True
```

- 2 If all three values show False, a hypervisor is probably already running and hiding them. Confirm with Task Manager → Performance → CPU: 'Virtualization: Enabled' means firmware is on.
- 3 On a Linux host, read the flags directly. vmx = Intel VT-x, svm = AMD-V; ept or npt = second-level translation.

```
$ grep -Eoc '(vmx|svm)' /proc/cpuinfo
$ grep -Eo 'ept|npt' /proc/cpuinfo | sort -u
```

EXPECTED OUTPUT

16
ept

- 4 If virtualization shows Disabled, reboot into firmware setup, enable Intel Virtualization Technology (or AMD SVM Mode) and VT-d/IOMMU if present, save, and repeat step 1.

CHECKPOINT VirtualizationFirmwareEnabled is True, or Task Manager shows Virtualization: Enabled.

- 5 Record your host spec sheet: CPU model, cores, threads, RAM, free SSD space, and each flag. Appendix A uses it to plan your RAM budget.

Check your work

- ☐ Virtualization enabled in firmware
- ☐ SLAT (EPT/RVI) supported
- ☐ Spec sheet completed

If it goes wrong

All values False but Task Manager says Enabled

Hyper-V or VBS owns the CPU. That is normal; Appendix A turns it off.

No virtualization option in firmware

Some consumer laptops hide it under Advanced or Security. Check the vendor's manual. Without it, use the Hands-on Labs path for every lab.

CLEAN UP None.

Stand up Workstation and a nested ESXi host

Run a Type 2 hypervisor on your laptop, install a Type 1 hypervisor inside it, and prove that host can run its own VMs.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
2–3 hrs	■ ■ ■ Intermediate	8 GB during install, 5 GB after	Lab 2 · Appendix A
PLATFORM	VMware Workstation Pro + ESXi 8 (nested)		

You will

- Install VMware Workstation Pro
- Install ESXi as a nested VM with virtualization passed through
- Adopt a snapshot-before-change habit

Before you start

- Complete Appendix A (host prep, downloads, virtual networks)
- ESXi 8.0U3e ISO from the Broadcom support portal

Procedure

- 1 Install Workstation Pro and add its folder to PATH so vmrun works from any prompt.

```
PS> [Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\Program Files (x86)\VMware\VMware Workstation", "User")
PS> vmrun
```

EXPECTED OUTPUT

```
vmrun version 1.17.0 build-...
Usage: vmrun [AUTHENTICATION-FLAGS] COMMAND [PARAMETERS]
```

- 2 Create a new VM: Custom → Guest OS 'VMware ESXi 8 and later' → 2 processors × 2 cores → 8 GB RAM → NAT network (vmnet8) → 40 GB disk. Name it esxi1 and store it in C:\Lab\esxi1.
- 3 Before powering on, edit settings: Processors → tick 'Virtualize Intel VT-x/EPT or AMD-V/RVI'. Add a second 80 GB thin disk for the datastore. Attach the ESXi ISO.
- 4 Power on and install ESXi to the 40 GB disk. Set a root password you will reuse across the lab.
- 5 At the yellow console screen, press F2 → Configure Management Network → IPv4 → static 192.168.80.11/24, gateway 192.168.80.2, DNS 192.168.80.2. Restart the management network.

CHECKPOINT The console shows <http://192.168.80.11/> and the page opens in your laptop's browser.

- 6 In the Host Client, go to Manage → Services and start TSM-SSH. Then confirm nested virtualization reached ESXi.

```
PS> ssh root@192.168.80.11
esxi# esxcfg-info | grep "HV Support"
```

```
EXPECTED OUTPUT
|---HV Support.....3
```

- 7 A value of 3 means VT is supported and enabled. Anything lower means the checkbox in step 3 is off, or Hyper-V is still running on your laptop.
- 8 Create datastore1 on the 80 GB disk (Storage → New datastore → VMFS 6). Shut ESXi down, reduce its RAM to 5 GB, and take your first snapshot.

```
PS> vmrun -T ws snapshot C:\Lab\esxi1\esxi1.vmx clean-install
```

Check your work

- ☐ Host Client reachable at 192.168.80.11
- ☐ HV Support = 3
- ☐ datastore1 exists
- ☐ Snapshot clean-install visible in Workstation's Snapshot Manager

If it goes wrong

'Virtualized Intel VT-x/EPT is not supported on this platform'

Hyper-V, VBS, or Memory Integrity is still on in Windows. Follow Appendix A step A2 and reboot.

Installer warns
HARDWARE_VIRTUALIZATION

Nested checkbox is off. Power off, tick it, reinstall is not required.

ESXi fails to boot at 4 GB

ESXi 8 needs 8 GB to install. After install it runs acceptably at 5 GB for this lab.

CLEAN UP Leave esxi1 powered off between labs to save RAM.

Build a VM deliberately, setting by setting

Create an Ubuntu Server VM in Workstation, justify every setting, install VMware Tools, and exercise every power state.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
90 min	 Foundation	2 GB		Lab 3
PLATFORM	VMware Workstation Pro			

You will

- Read a VM as a folder of files
- Choose firmware, controller, and NIC types on purpose
- See what shutdown, suspend, and power-off do to a guest

Before you start

- Ubuntu Server 24.04 ISO

Procedure

- 1 Before creating anything, write your choices and one-line reasons: 2 vCPUs, 2 GB RAM, 20 GB thin disk, UEFI firmware, Paravirtual SCSI controller, VMXNET3 NIC on NAT.
- 2 New Virtual Machine → Custom → 'I will install the operating system later' → Ubuntu 64-bit → Firmware: UEFI → 1 processor × 2 cores → 2048 MB → NAT → SCSI controller: Paravirtualized → New disk 20 GB, single file. Name it web01.
- 3 Set the NIC type by editing the .vmx while the VM is off, then look at every file in the folder.

```
PS> Add-Content C:\Lab\web01\web01.vmx 'ethernet0.virtualDev = "vmxnet3"'
PS> Get-ChildItem C:\Lab\web01 | Select Name, Length
```

EXPECTED OUTPUT

Name	Length	
web01.nvram	8684	
web01.vmdk	1245184	# thin: grows as the guest writes
web01.vmsd	0	# snapshot metadata
web01.vmx	2410	# the configuration: this IS the virtual hardware
web01.vmx.f	260	

- 4 Open web01.vmx in a text editor and find the lines for firmware, memory, CPU, disk controller, and NIC. Match each to your written choice.

CHECKPOINT You can point to `firmware = "efi"`, `memsize`, `numvcpus`, `scsi0.virtualDev = "pvscsi"`, and `ethernet0.virtualDev`.

- 5 Attach the ISO, power on, and install Ubuntu. In the installer's storage screen, note what hardware it reports.

```
EXPECTED OUTPUT
VMware Virtual disk 20.000G /dev/sda (via PVSCSI)
```

- 6 After first boot, install and check VMware Tools. Then compare mouse, clipboard, and display resize before and after.

```
guest$ sudo apt update && sudo apt install -y open-vm-tools
guest$ vmware-toolbox-cmd -v
guest$ lsmod | grep -E 'vmxnet3|vmw_pvscsi|vmw_balloon'
```

```
EXPECTED OUTPUT
12.3.5.46049 (build-22544099)
vmw_balloon          28672  0
vmxnet3              90112  0
vmw_pvscsi           32768  2
```

- 7 Exercise each power state from PowerShell. After each, note what the guest experienced.

```
PS> $vm = "C:\Lab\web01\web01.vmx"
PS> vmrun -T ws stop $vm soft      # asks the guest to shut down (needs Tools)
PS> vmrun -T ws start $vm nogui
PS> vmrun -T ws suspend $vm        # writes RAM to web01.vms, frees host RAM
PS> vmrun -T ws start $vm nogui    # resumes where it left off
PS> vmrun -T ws stop $vm hard      # pulls the virtual power cord
```

- 8 Start it once more and look for evidence of the hard stop.

```
guest$ journalctl --list-boots
guest$ journalctl -b -1 -n 5
```

```
EXPECTED OUTPUT
-- No entries --      # previous boot ended with no shutdown messages: an unclean stop
```

Check your work

- ☐ The `.vmx` lines match your written choices
- ☐ `vmware-toolbox-cmd` returns a version
- ☐ You can say in one line what each of the three power-off methods does

If it goes wrong

'vmrun stop soft' hangs or fails

VMware Tools not running yet. Check `systemctl status open-vm-tools`.

Stuck at the UEFI boot manager

No bootable disk yet. Attach the ISO and confirm 'Connect at power on' is ticked.

CLEAN UP Take a snapshot named base. web01 is reused through Lab 26.

Write a diagnostic plan before touching anything

Turn a vague fault report into a layered diagnostic plan that names the first layer to investigate and why.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
45 min	 Foundation	None		Lab 4
PLATFORM	Paper or text editor			

You will

- Apply define → locate layer → gather evidence → change one thing → evaluate
- Use blast radius to choose the starting layer
- Name the log that holds evidence at each layer

Before you start

- None. This lab is written, not executed.

Procedure

- 1 Read the scenario: 'At 09:10 users report the finance app on fin-app01 is slow. The same ESXi host runs 14 other VMs on one shared datastore. The nightly backup window ended at 09:00. Nobody changed anything.'
- 2 Write the problem as a measurable statement: what is slow, since when, compared to what baseline.
- 3 Blast radius: write the first check that tells you whether one VM or all 15 are affected.

CHECKPOINT Your check produces evidence without changing anything.
- 4 For each layer (guest OS, virtual hardware and VM config, host, shared storage and network), write the most likely cause given the scenario and the specific log or metric that would confirm it.
- 5 Rank the layers and justify your first choice in two sentences.
- 6 Write the single change you would make if your top hypothesis is confirmed, and how you would tell whether it worked.

Check your work

- ☐ Problem statement has a time and a measurable symptom
- ☐ Each layer has a hypothesis and a named evidence source
- ☐ The plan changes exactly one thing

If it goes wrong

Tempted to start with 'reboot the VM'

That destroys evidence and is a change, not a diagnosis.

CLEAN UP Keep the plan. You will compare it with your Lab 28 plan.

Virtualizing the Four Resources

CPU, memory, storage, and network: each one overcommitted, contended, and observed on purpose.

LAB 06

Chapter 6 · CPU Virtualization

Overcommit CPU and watch ready time appear

Assign more vCPUs than esxi1 has cores, generate contention, and read %RDY in esxtop.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
90 min	■ ■ ■ Intermediate	5 GB (esxi1)	Labs 3–4
PLATFORM	Nested ESXi (esxi1)		

You will

- Create a measurable overcommit ratio
- Read %RDY and %CSTP in esxtop
- Show that more vCPUs can make a VM slower

Before you start

- esxi1 at 5 GB with 4 logical CPUs
- One small Ubuntu Server VM on esxi1 named tiny01 (1 vCPU, 768 MB, 8 GB thin) with stress-ng and open-vm-tools installed

Procedure

- 1 Confirm esxi1's CPU count. This is your 'physical' core count for the lab.

```
esxi# esxcli hardware cpu global get
```

EXPECTED OUTPUT

```
CPU Cores: 4
CPU Packages: 1
CPU Threads: 4
```

- 2 Make three copies of tiny01 from the ESXi shell. This is a manual clone; Lab 12 covers the proper way.

```
esxi# cd /vmfs/volumes/datastore1
esxi# for i in 2 3 4; do
  mkdir tiny0$i
  vmkfstools -i tiny01/tiny01.vmdk -d thin tiny0$i/tiny0$i.vmdk
  sed -e "s/tiny01/tiny0$i/g" -e '/generatedAddress/d' -e '/uuid.bios/d' tiny01/tiny01.vmx >
tiny0$i/tiny0$i.vmx
  echo 'uuid.action = "create"' >> tiny0$i/tiny0$i.vmx
  vim-cmd solo/registervm /vmfs/volumes/datastore1/tiny0$i/tiny0$i.vmx
done
esxi# vim-cmd vmsvc/getallvms
```

- 3 In the Host Client, set tiny01–04 to 2 vCPUs each (8 vCPUs on 4 cores = 2:1). Power on tiny01 only and record a baseline.

```
guest$ stress-ng --cpu 2 --timeout 60s --metrics-brief
```

EXPECTED OUTPUT

```
stress-ng: info: [1203] stressor  bogo ops real time  usr time  sys time  bogo ops/s
stress-ng: info: [1203] cpu          24310    60.00    119.8     0.10    405.2
```

- 4 Open esxtop in a second SSH session and press c. Note %RDY for tiny01 at baseline.

```
esxi# esxtop
```

CHECKPOINT tiny01 %RDY is under 5% at baseline.

- 5 Power on tiny02–04 and start the same stress-ng in all four at once. Watch esxtop, then record tiny01's new bogo ops/s.

EXPECTED OUTPUT

GID	NAME	NWLD	%USED	%RUN	%SYS	%WAIT	%RDY	%CSTP
2210	tiny01	9	98.12	101.40	0.05	95.30	98.61	0.00
2245	tiny02	9	97.80	100.95	0.04	95.60	99.02	0.00

- 6 %RDY is summed across a VM's vCPUs. Divide by the vCPU count: about 49% per vCPU means each vCPU waited half the time for a core.
- 7 Now set tiny01 to 4 vCPUs, rerun with --cpu 4 under the same contention, and compare per-vCPU throughput and %CSTP with the 2-vCPU run. Record a table: vCPUs, contention on/off, bogo ops/s, %RDY per vCPU.

Check your work

- ☐ %RDY near 0 at baseline and high under contention
- ☐ Throughput dropped under contention
- ☐ 4-vCPU run shows equal or worse per-vCPU throughput, with %CSTP or %RDY higher

If it goes wrong

%RDY stays low	You are not overcommitted. Check total powered-on vCPUs exceeds esxi1's cores.
tiny02 asks 'Did you move or copy it?'	The uuid.action line is missing. Answer 'I copied it' in the Host Client.
Everything is slow, even at baseline	Your laptop is the bottleneck. Close other apps; nested ESXi only has what Workstation gives it.

CLEAN UP Stop the load and power off tiny02–04. Set tiny01 back to 1 vCPU.

Drive a host from ballooning into swap

Overcommit esxi1's memory and watch each reclaim technique engage in order, ending with host swapping.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
90 min	■■■ Advanced	5 GB (esxi1)	Lab 6
PLATFORM Nested ESXi (esxi1)			

You will

- Read the memory state in esxtop
- Observe ballooning, compression, and swapping
- Enable and measure page sharing between VMs

Before you start

- tiny01–04 from Lab 6, each with VMware Tools running (the balloon driver ships with it)
- Snapshot esxi1

Procedure

- 1 Set each tiny VM to 1.5 GB RAM (6 GB configured on a 5 GB host). Power on all four and let them idle.
- 2 Open esxtop and press m. Read the header and each VM's balloon (MCTLSZ), swap (SWCUR), and compression (CACHEUSD). Press f to add columns if needed.

```
esxi# esxtop
```

EXPECTED OUTPUT

```
PMEM /MB: 5119 total: 1026 vmk, 2890 other, 1203 free
VMKMEM/MB: 4987 managed: 299 minfree, 1180 rsvd, 3806 ursvd, high state
...
  GID NAME      MEMSZ  GRANT  MCTLSZ  SWCUR  CACHEUSD
  2210 tiny01   1536.00 612.00  0.00   0.00   0.00
```

- 3 Page sharing between VMs is off by default for security. Enable it for this lab, wait 10 minutes, and check the SHRD and ZERO columns.

```
esxi# esxcli system settings advanced set -o /Mem/ShareForceSalting -i 0
```

CHECKPOINT SHRD grows for idle VMs running the same OS.

- 4 In tiny01 and tiny02, allocate most of their memory and hold it.

```
guest$ stress-ng --vm 1 --vm-bytes 1200M --vm-keep --timeout 900s
```

- 5 Watch the state move from high toward soft. MCTLSZ rises on idle tiny03 and tiny04 as the balloon driver takes memory back from them.

```
EXPECTED OUTPUT
VMKMEM/MB: ... soft state
2280 tiny03 1536.00 402.00 620.00 0.00 0.00
2301 tiny04 1536.00 415.00 598.00 0.00 0.00
```

- 6 Start the same load in tiny03 and tiny04. Watch compression (CACHEUSD) and then host swap (SWCUR, SWR/s) appear. Time 'ls -R /usr > /dev/null' inside an idle VM during this phase.

```
EXPECTED OUTPUT
VMKMEM/MB: ... hard state
2210 tiny01 1536.00 1020.00 110.00 312.00 84.00
```

- 7 Record the order in which the techniques appeared and the state at each point.

Check your work

- ☐ SHRD rose while VMs were idle
- ☐ MCTLSZ rose on idle VMs under pressure
- ☐ SWCUR above 0 and guest response collapsed

If it goes wrong

MCTLSZ stays 0	VMware Tools (vmw_balloon) is not loaded. Check <code>lsmod grep vmw_balloon</code> .
----------------	---

A guest's OOM killer fires first	The balloon pushed the guest too far. Lower <code>--vm-bytes</code> .
----------------------------------	---

CLEAN UP Stop stress-ng, power off tiny02–04, set ShareForceSalting back to 2, and set each VM back to 768 MB.

Fill a thin-provisioned datastore on purpose

Compare thin and thick disks, then overcommit a small datastore until it fills and observe how VMs fail.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
90 min	■ ■ ■ Intermediate	5 GB (esxi1)	Lab 6
PLATFORM Nested ESXi (esxi1)			

You will

- Distinguish provisioned from used space
- Measure IOPS and latency with fio
- See ESXi pause a VM when thin storage runs out

Before you start

- Add a 6 GB disk to esxi1 in Workstation (hot-add or while off)
- fio installed in tiny01–03

Procedure

- 1 In the Host Client, create a new VMFS datastore named ds-small on the 6 GB disk.
- 2 Create one thin and one eager-zeroed thick disk and compare their real footprint.

```
esxi# cd /vmfs/volumes/ds-small
esxi# vmkfstools -c 2G -d thin thin.vmdk
esxi# vmkfstools -c 2G -d eagerzeroedthick thick.vmdk
esxi# ls -lh *-flat.vmdk; du -h *-flat.vmdk
```

EXPECTED OUTPUT

```
-rw----- 1 root root 2.0G thick-flat.vmdk
-rw----- 1 root root 2.0G thin-flat.vmdk
2.0G      thick-flat.vmdk
0         thin-flat.vmdk
```

- 3 Delete both test disks. Add a new 3 GB thin disk on ds-small to each of tiny01, tiny02, and tiny03. You have promised 9 GB from under 6 GB.

```
esxi# vmkfstools -U thin.vmdk; vmkfstools -U thick.vmdk
```

- 4 In tiny01, benchmark the new disk to get a baseline.

```
guest$ sudo fio --name=rr --filename=/dev/sdb --rw=randread --bs=4k --iodepth=32 --
ioengine=libaio --direct=1 --runtime=30 --time_based
```

EXPECTED OUTPUT

```
read: IOPS=18.2k, BW=71.1MiB/s (74.6MB/s)
clat (usec): min=112, max=9120, avg=1742.3
```

- 5 Open esxtop, press v (VM disk view) and note latency. Then in all three VMs, write until something breaks.

```
guest$ sudo dd if=/dev/urandom of=/dev/sdb bs=1M status=progress
esxi# watch -n 5 'df -h | grep ds-small'
```

CHECKPOINT You see ds-small's free space fall with each write.

- 6 When ds-small reaches 100%, ESXi pauses the VMs that need to write and asks a question. Read it.

```
esxi# vim-cmd vmsvc/getallvms | grep tiny
esxi# vim-cmd vmsvc/message <vmid>
```

EXPECTED OUTPUT

```
Virtual machine message _vmx1:
msg.hbacommon.outofspace:There is no more space for virtual disk '/vmfs/volumes/ds-
small/tiny02/tiny02.vmdk'. You might be able to continue this session by freeing disk space on
the relevant volume, and clicking Retry. Click Cancel to terminate this session.
0. Cancel
1. Retry
```

- 7 Do not answer yet. Record which VMs paused and what the guests saw. Then remove one VM's disk to free space and answer Retry on the others.

```
esxi# vim-cmd vmsvc/message <vmid> _vmx1 1
```

Check your work

- ☐ Thin file used far less than its provisioned size
- ☐ ds-small reached 100%
- ☐ At least one VM paused with the out-of-space question

If it goes wrong

No question appears

The VM may still be writing to cache. Wait, or check the Host Client's VM summary for a yellow question banner.

fio shows impossibly high IOPS

You forgot `--direct=1` and measured the guest's page cache.

CLEAN UP Remove the ds-small disks from each VM, delete ds-small, and remove the 6 GB disk from esxi1.

Connect two hosts to shared storage

Build a storage server, present NFS and iSCSI over a dedicated storage network, and mount both on two ESXi hosts.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■ ■ ■ Intermediate	11 GB	(two ESXi at 5 GB, stor1 at 1 GB)	Lab 8
PLATFORM esxi1 + esxi2 + a Linux storage VM				

You will

- Separate storage traffic onto its own network
- Mount the same datastore on two hosts
- Observe a lost storage path

Before you start

- Build esxi2 exactly like esxi1 (Lab 3) at 192.168.80.12
- Build stor1: Ubuntu Server, 1 vCPU, 1 GB, plus a 60 GB data disk
- vmnet2 (storage, 10.30.0.0/24) created per Appendix A

Procedure

- 1 Give stor1, esxi1, and esxi2 a second NIC on vmnet2. On stor1 set 10.30.0.10/24. On each ESXi host, add a VMkernel adapter on a new vSwitch1 bound to that NIC: esxi1 10.30.0.11, esxi2 10.30.0.12.

```
esxi# esxcli network vswitch standard add -v vSwitch1
esxi# esxcli network vswitch standard uplink add -v vSwitch1 -u vmnic1
esxi# esxcli network vswitch standard portgroup add -v vSwitch1 -p Storage
esxi# esxcli network ip interface add -i vmk1 -p Storage
esxi# esxcli network ip interface ipv4 set -i vmk1 -t static -I 10.30.0.11 -N 255.255.255.0
esxi# vmkping -I vmk1 10.30.0.10
```

EXPECTED OUTPUT

```
64 bytes from 10.30.0.10: icmp_seq=0 ttl=64 time=0.612 ms
```

- 2 On stor1, export a directory over NFS to the storage network only.

```
stor1$ sudo apt install -y nfs-kernel-server
stor1$ sudo mkdir -p /srv/vmstore && sudo chown nobody:nogroup /srv/vmstore
stor1$ echo "/srv/vmstore 10.30.0.0/24(rw,sync,no_subtree_check,no_root_squash)" | sudo tee
-a /etc/exports
stor1$ sudo exportfs -ra && sudo exportfs -v
```

- 3 Mount it on both hosts with the same datastore name.

```
esxi# esxcli storage nfs add -H 10.30.0.10 -s /srv/vmstore -v nas-nfs
esxi# esxcli storage nfs list
```

EXPECTED OUTPUT

Volume Name	Host	Share	Accessible	Mounted	Read-Only
nas-nfs	10.30.0.10	/srv/vmstore	true	true	false

CHECKPOINT nas-nfs shows Accessible true on both esxi1 and esxi2.

- 4 Create a file on nas-nfs from esxi1 and list it from esxi2. That is what shared storage means.

```
esxi1# touch /vmfs/volumes/nas-nfs/hello-from-esxi1
esxi2# ls /vmfs/volumes/nas-nfs/
```

- 5 Now present a block LUN over iSCSI. On stor1, create a 20 GB backstore and target with targetcli, allowing both ESXi initiators (read each host's IQN with esxcli iscsi adapter get -A vmhba65).

```
esxi# esxcli iscsi software set --enabled=true
esxi# esxcli iscsi adapter discovery sendtarget add -A vmhba65 -a 10.30.0.10:3260
esxi# esxcli storage core adapter rescan -A vmhba65
esxi# esxcli storage core device list | grep -A2 LIO
```

- 6 Format the LUN as VMFS once (from esxi1's Host Client, datastore name nas-iscsi) and rescan on esxi2 so it appears there too.
- 7 Move tiny01's files to nas-nfs (power off, move folder in the datastore browser, re-register). Power it on, start a ping inside it, then drop esxi1's storage uplink for 30 seconds in Workstation (Network Adapter 2 → uncheck Connected).

CHECKPOINT tiny01's I/O freezes during the outage and resumes when the link returns.

Check your work

- ☐ nas-nfs mounted on both hosts
- ☐ nas-iscsi visible on both hosts
- ☐ tiny01 runs from shared storage and survived a 30-second path loss

If it goes wrong

NFS mount fails: 'Unable to connect'	vmk1 cannot reach stor1. Test with vmkping -I vmk1 first.
iSCSI LUN never appears	stor1's ACL does not list the host's IQN, or targetcli config was not saved (saveconfig).
Two ESXi hosts don't fit in RAM	Lower both to 4.5 GB and power off web01. Or follow the Hands-on Labs path for Labs 11, 14, and 15.

CLEAN UP Keep esxi2, stor1, nas-nfs, and nas-iscsi. Labs 11–16 depend on them.

Wire VMs in bridged, NAT, and host-only modes

Attach VMs to each Workstation connection mode, predict reachability, test it, then segment traffic with a VLAN inside ESXi.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
75 min	 Foundation	4 GB		Lab 4
PLATFORM	VMware Workstation Pro + esxi1			

You will

- Use bridged, NAT, and host-only networks
- Predict reachability before testing
- Tag VM traffic onto a VLAN with a port group

Before you start

- web01 and a linked clone named net-b (right-click web01 → Manage → Clone → linked)

Procedure

- 1 Open Edit → Virtual Network Editor and note the three defaults: vmnet0 (bridged to your physical adapter), vmnet1 (host-only), vmnet8 (NAT). Record each subnet.
- 2 Before testing, fill a reachability matrix: from a VM in each mode, can it reach (a) your laptop, (b) the other VM, (c) another device on your home network, (d) the internet?
- 3 Put web01 on NAT and net-b on bridged. Test each target with ping and curl.

```
guest$ ip -br addr
guest$ ping -c2 <laptop-ip>; ping -c2 <other-vm-ip>; ping -c2 <router-ip>; curl -sI
https://example.com | head -1
```

- 4 Move net-b to host-only (vmnet1) and repeat.

EXPECTED OUTPUT

```
ping: connect: Network is unreachable      # no internet from host-only
```

CHECKPOINT Your measured matrix matches your prediction, or you can explain each difference.

- 5 On esxi1, create two port groups on vSwitch0: VLAN0 (ID 0) and VLAN40 (ID 40). Put tiny01 on VLAN0 and tiny02 on VLAN40.

```
esxi# esxcli network vswitch standard portgroup add -v vSwitch0 -p VLAN40
esxi# esxcli network vswitch standard portgroup set -p VLAN40 --vlan-id 40
esxi# esxcli network vswitch standard portgroup list
```

EXPECTED OUTPUT

Name	Virtual Switch	Active Clients	VLAN ID
Management Network	vSwitch0	1	0
VM Network	vSwitch0	2	0
VLAN40	vSwitch0	1	40

- 6 Give both VMs static addresses in the same subnet and ping between them. Then move tiny02 back to VM Network and ping again.

Check your work

- ☐ Matrix completed for all three modes
- ☐ Host-only VM reaches only the laptop and its peer
- ☐ VMs on different VLAN port groups cannot reach each other

If it goes wrong

Bridged VM gets no address

You are on a network that blocks unknown MACs (common on corporate and hotel Wi-Fi). Use NAT.

Bridged to the wrong adapter

In the Virtual Network Editor, set vmnet0 to your active adapter instead of Automatic.

CLEAN UP Return every VM to NAT. Delete the VLAN40 port group.

Separate traffic types across two hosts

Build identical per-host networking by hand, see how small inconsistencies break things, then manage the same design centrally in a distributed switch.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■■■ Advanced	11 GB		Lab 9
PLATFORM esxi1 + esxi2 · Hands-on Labs for the distributed switch				

You will

- Separate management, migration, storage, and VM traffic
- See why per-host configuration drifts
- Configure a vSphere Distributed Switch

Before you start

- esxi1, esxi2, stor1 from Lab 9
- vmnet3 (migration, 10.20.0.0/24) per Appendix A
- A free Broadcom account for VMware Hands-on Labs

Procedure

- 1 Add a third NIC on vmnet3 to each ESXi host. Create vSwitch2 with a vMotion VMkernel port and tag it for vMotion.

```
esxi# esxcli network vswitch standard add -v vSwitch2
esxi# esxcli network vswitch standard uplink add -v vSwitch2 -u vmnic2
esxi# esxcli network vswitch standard portgroup add -v vSwitch2 -p vMotion
esxi# esxcli network ip interface add -i vmk2 -p vMotion
esxi# esxcli network ip interface ipv4 set -i vmk2 -t static -I 10.20.0.11 -N 255.255.255.0
esxi# esxcli network ip interface tag add -i vmk2 -t VMotion
esxi# esxcli network ip interface tag get -i vmk2
```

EXPECTED OUTPUT

Tags: VMotion

- 2 Print each host's traffic map and confirm they match.

```
esxi# esxcli network ip interface list | grep -E 'Name|Portgroup'
esxi# esxcli network vswitch standard portgroup list
```

CHECKPOINT Both hosts have identical port group names on vSwitch0–2.

- 3 Introduce drift on purpose: on esxi2 only, rename 'VM Network' to 'VM-Network'. Try to register a VM from nas-nfs on esxi2 that expects 'VM Network' and power it on.

EXPECTED OUTPUT

The virtual machine's network adapter "Network adapter 1" is connected to a network that is unavailable.

- 4 Undo the rename. Write two sentences: what would this drift do to migration or HA across 20 hosts?
- 5 Open VMware Hands-on Labs, search the catalog for the current vSphere introductory lab, and start it. Find the distributed switch in the Networking view.
- 6 In the Hands-on Lab, create a distributed port group, set its VLAN, and add a VMkernel adapter for vMotion on every host in one wizard. Note where teaming and Network I/O Control are configured once for all hosts.

Check your work

- ☐ vmk2 tagged VMotion on both hosts
- ☐ Drift reproduced and explained
- ☐ Distributed port group created in Hands-on Labs

If it goes wrong

vmk2 cannot ping the other host	Both vmnet3 adapters must be on the same Workstation network; check the Virtual Network Editor.
Hands-on Lab takes long to start	Labs can take several minutes to provision. Request it before you start the local steps.

CLEAN UP Keep vSwitch2 and vmk2; Lab 14 uses them.

Operating the Platform

Snapshots, templates, resource controls, migration, clustering, backup, and automation.

LAB 12

Chapter 12 · Snapshots, Clones, and Templates

Snapshot chains, a golden template, and custom clones

Work a snapshot chain safely, turn a prepared VM into a template, and deploy linked and full clones that get unique identities on first boot.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■ ■ ■ Intermediate	4 GB		Lab 4
PLATFORM VMware Workstation Pro + cloud-init				

You will

- Take, revert, and delete snapshots and see the delta disks
- Generalize a golden image
- Customize clones on deployment with cloud-init

Before you start

- web01 with snapshot base
- An SSH key pair on your laptop (ssh-keygen -t ed25519)

Procedure

- 1 Build a three-deep chain, making a visible change before each snapshot (create a file, install a package, create a user).

```
PS> $vm = "C:\Lab\web01\web01.vmx"
PS> vmrun -T ws snapshot $vm s1; vmrun -T ws snapshot $vm s2; vmrun -T ws snapshot $vm s3
PS> vmrun -T ws listSnapshots $vm showTree
PS> Get-ChildItem C:\Lab\web01\*.vmdk | Select Name, Length
```

```

EXPECTED OUTPUT
Total snapshots: 4
base
    s1
        s2
            s3
web01.vmdk          5.1 GB
web01-000001.vmdk   84 MB    # each snapshot adds a delta disk
web01-000002.vmdk   112 MB
web01-000003.vmdk   41 MB

```

- 2 Revert to s1 and confirm the later changes are gone. Then delete s2 and s3 and watch the delta files disappear.

```

PS> vmrun -T ws revertToSnapshot $vm s1
PS> vmrun -T ws deleteSnapshot $vm s3; vmrun -T ws deleteSnapshot $vm s2

```

CHECKPOINT Only web01.vmdk and one delta remain.

- 3 Build the golden image: full-clone web01 to golden (VM → Manage → Clone → full). Boot it, patch it, then generalize so clones get unique identities.

```

guest$ sudo apt update && sudo apt -y full-upgrade && sudo apt install -y cloud-init
guest$ sudo cloud-init clean --logs --machine-id
guest$ sudo rm -f /etc/ssh/ssh_host_*
guest$ sudo poweroff

```

- 4 Snapshot golden as 'sealed' and mark it as a template: VM → Settings → Options → Advanced → tick 'Enable Template mode'.
- 5 Write cloud-init data for one clone. cloud-init reads it from the VM's configuration through VMware Tools' guestinfo channel.

```

# app01-user.yaml
#cloud-config
hostname: app01
users:
  - name: lab
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys: [ "<paste your public key>" ]

```

- 6 Create a linked and a full clone, then inject the data into the linked clone's .vmx.

```

PS> vmrun -T ws clone C:\Lab\golden\golden.vmx C:\Lab\app01\app01.vmx linked -
snapshot=sealed -cloneName=app01
PS> vmrun -T ws clone C:\Lab\golden\golden.vmx C:\Lab\app02\app02.vmx full -snapshot=sealed
-cloneName=app02
PS> $ud = [Convert]::ToBase64String([IO.File]::ReadAllBytes("app01-user.yaml"))
PS> Add-Content C:\Lab\app01\app01.vmx "guestinfo.userdata = `"$ud`""
PS> Add-Content C:\Lab\app01\app01.vmx 'guestinfo.userdata.encoding = "base64"'
PS> Add-Content C:\Lab\app01\app01.vmx 'guestinfo.metadata = "e30="'
PS> Add-Content C:\Lab\app01\app01.vmx 'guestinfo.metadata.encoding = "base64"'

```

7 Compare the two clone folders' sizes. Boot app01 and SSH in with your key.

```

PS> "{0:N0} MB" -f ((Get-ChildItem C:\Lab\app01 -Recurse | Measure Length -Sum).Sum/1MB)
PS> "{0:N0} MB" -f ((Get-ChildItem C:\Lab\app02 -Recurse | Measure Length -Sum).Sum/1MB)
guest$ hostnamectl --static; cat /etc/machine-id; cloud-init status

```

EXPECTED OUTPUT

```

38 MB          # linked clone
5,212 MB       # full clone
app01
7c1f0e9a4b2d4f6f8e3a0b1c2d3e4f50
status: done

```

Check your work

- ☐ Revert removed later changes and deleting snapshots removed delta files
- ☐ Linked clone is a small fraction of the full clone
- ☐ app01 booted with its own hostname, machine-id, and your SSH key

If it goes wrong

Cannot delete golden	Linked clones depend on its sealed snapshot. That is the parent-dependency lesson; full-clone or delete them first.
cloud-init status: disabled	The VMware datasource needs open-vm-tools and the guestinfo keys. Check with <code>vmware-rpctool 'info-get guestinfo.userdata'</code> .
Clones get the same DHCP address	machine-id was not cleared before sealing.

CLEAN UP Keep golden and app01. Delete app02.

Shares, limits, reservations, and finding the bottleneck

Use resource pools and per-VM controls to prioritize VMs under contention, then identify a hidden bottleneck from host metrics alone.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■■■ Advanced	5 GB		Labs 6–8
PLATFORM	Nested ESXi (esxi1)			

You will

- Create resource pools with different shares
- See that shares only matter under contention
- Name the constrained resource from esxtop

Before you start

- tiny01 and tiny02 on esxi1
- Both at 1 vCPU

Procedure

- 1 In the Host Client, create two resource pools under the host: Gold (CPU shares High) and Bronze (CPU shares Low). Move tiny01 into Gold and tiny02 into Bronze.
- 2 Pin both VMs to the same physical CPU so they must compete: VM → Edit → Advanced → Configuration Parameters, add sched.cpu.affinity = 0. Power both on.
- 3 Run the same CPU load in both at once and compare throughput.

```
guest$ stress-ng --cpu 1 --timeout 60s --metrics-brief
```

EXPECTED OUTPUT

```
tiny01 (Gold, High):  bogo ops/s  318.4
tiny02 (Bronze, Low): bogo ops/s   80.1      # High:Low shares = 4:1
```

CHECKPOINT The throughput ratio is close to 4:1.

- 4 Stop tiny01's load and rerun tiny02 alone. Write down why its throughput recovered even though its shares are still Low.
- 5 Now cap tiny02 with a limit of 500 MHz (Edit → CPU → Limit) and rerun alone. Then give tiny01 a memory reservation of 512 MB and look at the .vswp file size in its folder.

```
esxi# ls -lh /vmfs/volumes/nas-nfs/tiny01/*.vswp
```

EXPECTED OUTPUT

```
-rw----- 1 root  root  256.0M  tiny01-a1b2c3d4.vswp      # 768 MB configured - 512 MB reserved
```

- 6 Remove the limit, reservation, and affinity. Ask a partner (or use the blind method in Lab 28) to start one hidden load on a tiny VM: CPU, memory, disk, or network. Using esxtop only, identify which.

```
esxi# esxtop      # c: %RDY/%USED  m: MCTLSZ/SWCUR  v: LAT/rd, LAT/wr  n: MbTX/s, MbRX/s
```

- 7 Write the esxtop view, column, and value that gave it away.

Check your work

- ☐ Shares produced ~4:1 under contention and no effect without it
- ☐ Limit held tiny02 to about 500 MHz even with idle capacity
- ☐ Hidden load correctly named from esxtop

If it goes wrong

No difference between pools

The VMs are not competing. Check that affinity was applied (power-cycle after adding it).

Affinity option missing

Affinity is not available when a VM is in a DRS cluster. On standalone esxi1 it should be present.

CLEAN UP Remove affinity, limits, and reservations. Keep the pools.

Migrate cold locally, live in the Hands-on Lab, then break it

Move a VM between hosts using shared storage, do a live migration without dropping a ping, and diagnose three blocked migrations.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■ ■ ■ Intermediate	11 GB		Labs 9, 11
PLATFORM esxi1 + esxi2 · Hands-on Labs for vMotion				

You will

- Perform a cold migration by hand
- Perform vMotion and Storage vMotion
- Map each failure to its missing requirement

Before you start

- tiny01 on nas-nfs
- esxi2 powered on
- A running vSphere Hands-on Lab

Procedure

- 1 Cold migration by hand: power off tiny01 on esxi1, unregister it, and register the same files on esxi2. Nothing is copied; shared storage makes this instant.

```
esxi1# vim-cmd vmsvc/getallvms | grep tiny01
esxi1# vim-cmd vmsvc/unregister <vmid>
esxi2# vim-cmd solo/registervm /vmfs/volumes/nas-nfs/tiny01/tiny01.vmx
esxi2# vim-cmd vmsvc/power.on <new-vmid>
```

- 2 Cold storage migration: power off, copy the disk to esxi2's local datastore, and point the VM at it.

```
esxi2# vmkfstools -i /vmfs/volumes/nas-nfs/tiny01/tiny01.vmdk -d thin
/vmfs/volumes/datastore1/tiny01/tiny01.vmdk
```

CHECKPOINT tiny01 boots on esxi2 from local storage. Move it back to nas-nfs before continuing.

- 3 In the Hands-on Lab, start a continuous ping to a running VM from the lab's console, then vMotion it to another host (right-click → Migrate → Change compute resource only).
- 4 Read the task's result and count dropped pings.

EXPECTED OUTPUT

```
Reply from 192.168.110.52: bytes=32 time<1ms TTL=64
Reply from 192.168.110.52: bytes=32 time=3ms TTL=64      # switchover
Reply from 192.168.110.52: bytes=32 time<1ms TTL=64
```

- 5 In the Hands-on Lab, do a Storage vMotion (Change storage only) while the VM runs.
- 6 Break 1, local resource: attach an ISO from a host's local datastore to the VM and try to vMotion. Read the compatibility check.
- 7 Break 2, network: disconnect the VM's NIC from a port group that exists on the target and connect it to one that exists only on the source. Try again. Break 3, CPU: note where the wizard reports CPU compatibility, and look up Enhanced vMotion Compatibility (EVC) in the cluster settings. For each break, record the exact message and the requirement it maps to.

Check your work

- ☐ Cold migration done locally in under a minute
- ☐ vMotion completed with at most one dropped ping
- ☐ Three blocked migrations recorded and mapped to storage, network, and CPU

If it goes wrong

registervm fails: 'already exists'	The VM is still registered on esxi1. Unregister it there first.
------------------------------------	---

Power-on asks 'moved or copied?'	Answer 'I moved it' so the VM keeps its UUID and MAC.
----------------------------------	---

CLEAN UP Detach the ISO and restore the NIC in the Hands-on Lab. Locally, return tiny01 to esxi1.

Pull the plug on a host

Do by hand what HA does automatically, then enable vSphere HA and watch it restart VMs after a host failure.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■■■ Advanced	11 GB		Lab 14

PLATFORM esxi1 + esxi2 · Hands-on Labs for vSphere HA

You will

- Understand file locking on shared storage
- Recover a VM manually after host loss
- Configure and observe vSphere HA and admission control

Before you start

- tiny01 running on esxi1 from nas-nfs
- A running vSphere Hands-on Lab

Procedure

- 1 Start a ping to tiny01 from your laptop. Predict how long the outage will be if you recover by hand.
- 2 Kill esxi1: in Workstation, choose Power Off (not shut down) for esxi1.

CHECKPOINT The ping to tiny01 stops.

- 3 Try to register and power on tiny01 on esxi2 immediately. Read the error.

```
esxi2# vim-cmd solo/registervm /vmfs/volumes/nas-nfs/tiny01/tiny01.vmx
esxi2# vim-cmd vmsvc/power.on <vmid>
```

EXPECTED OUTPUT

Power on failed: Failed to lock the file # esxi1's lock has not expired yet

- 4 Wait about a minute and retry. Answer 'I moved it' if prompted. Record the total outage time.
- 5 Write down what would happen if esxi1 were only isolated (still alive and writing) and you started the VM on esxi2 anyway. This is split-brain.
- 6 In the Hands-on Lab, open the cluster's settings and turn on vSphere HA. Read the heartbeat datastores and admission control settings.

- 7 If the lab lets you, simulate a host failure (or place a host into maintenance mode to see DRS move VMs). Watch Monitor → vSphere HA and the event log, and compare the recovery with your manual time.

Check your work

- ☐ Manual recovery succeeded and was timed
- ☐ You can explain the lock error
- ☐ HA configured in Hands-on Labs; admission control setting recorded

If it goes wrong

Both hosts show tiny01	You registered it twice. Unregister it from the host that is not running it.
Lock never releases	The lock holder is still alive. Check esxi1 is really powered off.

CLEAN UP Power esxi1 back on. Unregister tiny01 from whichever host should not own it.

Back up, delete, restore, and prove it boots

Take a snapshot-based image backup of a running VM, destroy the VM, restore it, and verify its data survived.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
90 min	■ ■ ■ Intermediate	5 GB	Labs 9, 12
PLATFORM	esxi1 + ghettovcb (community script) or Workstation export		

You will

- Run an image-level backup of a running VM
- Compare crash-consistent and application-consistent backups
- Complete and time a full restore

Before you start

- tiny01 on esxi1 with VMware Tools
- A backup folder on nas-nfs:
/vmfs/volumes/nas-nfs/backups
- ghettovcb (open-source, from GitHub) copied to esxi1; it needs ESXi in evaluation mode or a licensed API

Procedure

- 1 Inside tiny01, create data you can verify later.

```
guest$ sudo apt install -y sqlite3
guest$ sqlite3 ~/lab.db "create table t(x); insert into t values('before-backup');"
guest$ sha256sum ~/lab.db | tee ~/lab.sha
```

- 2 Install ghettovcb on esxi1 and configure it to write to nas-nfs.

```
esxi# esxcli software vib install -v /tmp/vghetto-ghettoVCB.vib -f
esxi# sed -i 's#^VM_BACKUP_VOLUME=.*#VM_BACKUP_VOLUME=/vmfs/volumes/nas-nfs/backups#'
/etc/ghettovcb/ghettoVCB.conf
esxi# sed -i 's#^VM_SNAPSHOT_QUIESCE=.*#VM_SNAPSHOT_QUIESCE=1#'
/etc/ghettovcb/ghettoVCB.conf
```

- 3 Back up tiny01 while it runs. It takes a quiesced snapshot, copies the frozen base disk, then removes the snapshot.

```
esxi# /opt/ghettovcb/bin/ghettoVCB.sh -m tiny01 -g /etc/ghettovcb/ghettoVCB.conf
```

EXPECTED OUTPUT

```
Initiate backup for tiny01
Creating Snapshot "ghettoVCB-snapshot-2026-09-24" for tiny01
Destination disk format: VMDK thin
Removing snapshot from tiny01 ...
Successfully completed backup for tiny01!
```

CHECKPOINT A dated folder exists under nas-nfs/backups/tiny01.

4 Write one sentence on why a database needs more than a filesystem quiesce to be application-consistent.

5 Destroy the VM. Really.

```
esxi# vim-cmd vmsvc/power.off <vmid>; vim-cmd vmsvc/destroy <vmid>
```

6 Restore from the backup folder and time it.

```
esxi# echo '"vmfs/volumes/nas-nfs/backups/tiny01/tiny01-2026-09-24_10-00-00;/vmfs/volumes/datastore1;1"' > /tmp/restore.txt
esxi# time /opt/ghettovcb/bin/ghettoVCB-restore.sh -c /tmp/restore.txt
```

7 Register and boot the restored VM, then verify the checksum and the row. Write your measured RPO (how much data you could lose) and RTO (how long recovery took).

```
guest$ sha256sum -c ~/lab.sha && sqlite3 ~/lab.db "select * from t;"
```

EXPECTED OUTPUT

```
/home/lab/lab.db: OK
before-backup
```

Check your work

- ☐ Backup completed with a quiesced snapshot
- ☐ VM restored and boots cleanly
- ☐ Checksum matches and the row is present
- ☐ RPO and RTO written down

If it goes wrong

Backup fails with an API or license error

The free ESXi license makes the API read-only. Run this lab inside the 60-day evaluation, or use the fallback: power off the VM and export it with ovftool.

Quiesce fails

VMware Tools is not running in the guest.

CLEAN UP Keep one backup. Delete older test backups.

Provision a fleet from a definition file

Write a PowerCLI script that reads a definition file and creates, configures, and starts several identical VMs, reproducibly.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2–3 hrs	■■■ Advanced	5 GB		Lab 12
PLATFORM	PowerCLI against esxi1			

You will

- Connect to the ESXi API with PowerCLI
- Drive VM creation from a data file
- Prove the script is idempotent

Before you start

- esxi1 in evaluation mode (write API required)
- PowerShell 7 on your laptop

Procedure

- 1 Install PowerCLI and connect.

```
PS> Install-Module VCF.PowerCLI -Scope CurrentUser
PS> Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -Confirm:$false
PS> Connect-VIServer 192.168.80.11 -User root
PS> Get-VMHost | Select Name, ConnectionState, NumCpu, MemoryTotalGB
```

```
EXPECTED OUTPUT
Name                ConnectionState NumCpu MemoryTotalGB
----                -
192.168.80.11      Connected         4      5.000
```

- 2 Write the definition file.

```
# fleet.csv
Name,NumCpu,MemoryGB,DiskGB,Network
web-a,1,0.5,4,VM Network
web-b,1,0.5,4,VM Network
web-c,1,0.5,4,VM Network
```

- 3 Write provision.ps1. It must skip VMs that already exist.

```
$fleet = Import-Csv fleet.csv
foreach ($d in $fleet) {
    if (Get-VM -Name $d.Name -ErrorAction SilentlyContinue) { "skip $($d.Name)"; continue }
    New-VM -Name $d.Name -Datastore datastore1 -NumCpu $d.NumCpu `
        -MemoryGB $d.MemoryGB -DiskGB $d.DiskGB -DiskStorageFormat Thin `
        -NetworkName $d.Network -GuestId ubuntu64Guest | Out-Null
    New-TagAssignment -Entity $d.Name -Tag fleet -ErrorAction SilentlyContinue
    "created $($d.Name)"
}
```

4 Run it, then run it again.

```
PS> ./provision.ps1; ./provision.ps1
```

EXPECTED OUTPUT

```
created web-a
created web-b
created web-c
skip web-a
skip web-b
skip web-c
```

CHECKPOINT The second run changes nothing.

- 5 Delete web-b in the Host Client, re-run the script, and confirm it rebuilds exactly that one VM. That is treating VMs as cattle.
- 6 See the API underneath: every PowerCLI cmdlet is a call to the same API the Host Client uses. Open <https://192.168.80.11/mob> and browse content → rootFolder → childEntity to find web-a.
- 7 Extend the script: for each VM, clone from a template disk instead of an empty disk, and pass cloud-init data with New-AdvancedSetting guestinfo.userdata (as in Lab 12).

Check your work

- ☐ Three VMs created from one command
- ☐ Second run reports only skips
- ☐ Deleted VM rebuilt identically

If it goes wrong

'Current license or ESXi version prohibits execution'

The free license makes the API read-only. Use the evaluation period, or run the same script in a Hands-on Lab against vCenter.

New-TagAssignment fails

Tags need vCenter. Remove that line on a standalone host.

```
CLEAN UP Get-VM web-* | Remove-VM -DeletePermanently -Confirm:$false
```

The Major Platforms

vSphere, Hyper-V, and KVM/Proxmox mapped back to the concepts, plus desktop delivery.

LAB 18

Chapter 18 · VMware vSphere

Map every concept onto vSphere

Tour the vSphere architecture from the ESXi shell up to vCenter and build a concept-to-feature map from everything you have done so far.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
3 hrs	■ ■ ■ Intermediate	5 GB		Labs 3–17
PLATFORM	esxi1 + Hands-on Labs (vCenter)			

You will

- Find ESXi's logs and tools
- Understand what vCenter adds
- Map course concepts to vSphere names

Before you start

- esxi1 powered on
- A running vSphere Hands-on Lab

Procedure

- 1 On esxi1, inventory the host from the shell.

```
esxi# vmware -v1
esxi# esxcli system version get
esxi# esxcli software vib list | wc -l
esxi# ls /var/log/
```

EXPECTED OUTPUT

```
VMware ESXi 8.0.3 build-24677879
VMware ESXi 8.0 Update 3e
...
hostd.log vmkernel.log vpxa.log fdm.log shell.log auth.log ...
```

- 2 Tail the two logs you will use most while powering a VM on and off.

```
esxi# tail -f /var/log/hostd.log /var/log/vmkernel.log
```

CHECKPOINT You can find the power-on task in hostd.log.

- 3 Read one VM's own log, which lives next to its disk.

```
esxi# tail -50 /vmfs/volumes/datastore1/tiny02/vmware.log
```

- 4 In the Hands-on Lab, find the vCenter components: inventory, tasks and events, alarms, content library, and the cluster settings (HA, DRS, EVC).
- 5 Build the concept map as a table with three columns: concept, vSphere feature, where you saw it.
- 6 Note which features the free ESXi license and a standalone host cannot use.

Check your work

- ☐ Located hostd.log, vmkernel.log, and a VM's vmware.log
- ☐ Concept map has at least twelve rows

If it goes wrong

Evaluation expired

Apply the free license key from the download page. The Host Client keeps working; API writes (Labs 16–17) stop.

CLEAN UP None.

Hyper-V VMs, a switch, and replica to a second host

Enable Hyper-V, build a VM and virtual switch in PowerShell, and replicate the VM to a second host using certificate authentication.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
3 hrs	■■■ Advanced	9 GB (hv1 and hv2 at 4.5 GB)	Lab 3
PLATFORM Two nested Windows Server VMs in Workstation			

You will

- Install the Hyper-V role
- Configure dynamic memory on a Generation 2 VM
- Set up and test Hyper-V Replica without a domain

Before you start

- Power off ESXi hosts to free RAM
- Two Windows Server 2025 evaluation VMs, hv1 and hv2, on NAT, each with 'Virtualize Intel VT-x/EPT' ticked
- Hosts file on each with the other's name and IP

Procedure

- 1 Install the role on both hosts.

```
PS> Install-WindowsFeature -Name Hyper-V -IncludeManagementTools -Restart
```

EXPECTED OUTPUT

Success	Restart Needed	Exit Code	Feature Result
True	Yes	SuccessRest...	{Hyper-V, Hyper-V Module for Windows PowerShell...}

- 2 Create an external switch and a small Generation 2 VM with dynamic memory.

```
PS> New-VMSwitch -Name "Lab-External" -NetAdapterName "Ethernet0" -AllowManagementOS $true
PS> New-VM -Name hv-web01 -Generation 2 -MemoryStartupBytes 512MB -NewVHDPATH C:\VMs\hv-web01.vhdx -NewVHDSIZE 4GB -SwitchName "Lab-External"
PS> Set-VMemory hv-web01 -DynamicMemoryEnabled $true -MinimumBytes 256MB -MaximumBytes 1GB
PS> Get-VM hv-web01 | Select Name, State, Generation, DynamicMemoryEnabled
```

- 3 Create a self-signed certificate on each host, valid for both server and client authentication, and trust the other host's certificate.

```
PS> $c = New-SelfSignedCertificate -DnsName $env:COMPUTERNAME -CertStoreLocation
Cert:\LocalMachine\My -KeyUsage DigitalSignature,KeyEncipherment -TextExtension
@("2.5.29.37={text}1.3.6.1.5.5.7.3.1,1.3.6.1.5.5.7.3.2")
PS> Export-Certificate -Cert $c -FilePath C:\$env:COMPUTERNAME.cer
# copy the .cer to the other host, then on that host:
PS> Import-Certificate -FilePath C:\hv1.cer -CertStoreLocation Cert:\LocalMachine\Root
PS> New-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows
NT\CurrentVersion\Virtualization\Replication" -Name DisableCertRevocationCheck -Value 1 -
PropertyType DWord -Force
```

- 4 On hv2, enable it as a replica server over HTTPS and open the firewall.

```
PS> Set-VMReplicationServer -ReplicationEnabled $true -AllowedAuthenticationType Certificate
-CertificateThumbprint <hv2-thumbprint> -ReplicationAllowedFromAnyServer $true -
DefaultStorageLocation C:\Replica
PS> Enable-NetFirewallRule -DisplayName "Hyper-V Replica HTTPS Listener (TCP-In)"
```

- 5 On hv1, enable replication and send the initial copy.

```
PS> Enable-VMReplication -VMName hv-web01 -ReplicaServerName hv2 -ReplicaServerPort 443 -
AuthenticationType Certificate -CertificateThumbprint <hv1-thumbprint>
PS> Start-VMInitialReplication -VMName hv-web01
PS> Measure-VMReplication
```

EXPECTED OUTPUT

Name	State	Health	LReplTime	PReplSize(M)	AvgLatency	AvgReplSize(M)
Relationship	-----	-----	-----	-----	-----	-----

hv-web01	Replicating	Normal	9/24/2026 10:42:10	0.00	00:00:01	0.00
						Simple

CHECKPOINT Health reads Normal.

- 6 Run a test failover on hv2, confirm the test VM appears, then read the Hyper-V event log.

```
PS> Start-VMFailover -VMName hv-web01 -AsTest
PS> Get-VM "hv-web01 - Test"
PS> Get-WinEvent -LogName Microsoft-Windows-Hyper-V-VMMS-Admin -MaxEvents 10 | Format-Table
TimeCreated, Id, Message -Wrap
```

Check your work

- ☐ hv-web01 exists with dynamic memory
- ☐ Replication health Normal
- ☐ Test failover VM created on hv2

If it goes wrong

'Hyper-V cannot be installed: the processor does not have required virtualization capabilities'

The nested checkbox is off on hv1/hv2 in Workstation.

Replication fails with a certificate error

Wrong thumbprint, missing Root import, or revocation check still enabled.

No OS in hv-web01

Not required. Replica copies the VHDX regardless of what is on it.

CLEAN UP On hv2: Stop-VMFailover -VMName hv-web01. Power off hv1 and hv2 when done.

A two-node Proxmox cluster with live migration

Build a small Proxmox cluster, see KVM and QEMU underneath it, and live-migrate a VM between nodes.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■ ■ ■ Intermediate	7 GB	(pve1 and pve2 at 3.5 GB)	Lab 3
PLATFORM	Two nested Proxmox VE VMs in Workstation			

You will

- Install Proxmox VE nested
- Identify the KVM module, the QEMU process, and the management layer
- Live-migrate between two nodes

Before you start

- Power off ESXi and Hyper-V VMs
- Proxmox VE ISO

Procedure

- 1 Create two Workstation VMs (2 vCPU, 3.5 GB, 32 GB disk, NAT, nested ticked). Install Proxmox as pve1 (192.168.80.21) and pve2 (192.168.80.22).

- 2 Confirm KVM is available inside each node.

```
pve1# lsmod | grep kvm
pve1# ls -l /dev/kvm
```

EXPECTED OUTPUT

```
kvm_intel          409600  0
kvm                1363968  1 kvm_intel
```

- 3 Create the cluster on pve1 and join pve2.

```
pve1# pvecm create lab
pve2# pvecm add 192.168.80.21
pve1# pvecm status | grep -E 'Nodes|Quorate'
```

EXPECTED OUTPUT

```
Nodes:          2
Quorate:        Yes
```

- 4 Create a tiny VM (100) on pve1. It can boot a small ISO such as Alpine. Then look at the process that is the VM.

```
pve1# qm start 100
pve1# ps -o pid,rss,cmd -C kvm | cut -c1-160
pve1# cat /etc/pve/qemu-server/100.conf
```

CHECKPOINT You can match the -m and -smp flags on the process to the .conf file.

- 5 Live-migrate the running VM, copying its local disk.

```
pve1# qm migrate 100 pve2 --online --with-local-disks
```

```
EXPECTED OUTPUT
... migration active, transferred 312.4 MiB of 1.0 GiB VM-state
... migration status: completed
... migration finished successfully (duration 00:00:41)
```

- 6 Optional libvirt tour: in a Ubuntu VM with nested enabled, install libvirt and compare it with Proxmox.

```
guest$ sudo apt install -y qemu-kvm libvirt-daemon-system virtinst
guest$ virsh list --all; virsh capabilities | grep -m1 '<model>'
```

- 7 Write a three-row table: KVM, QEMU, and libvirt/Proxmox, and what each does.

Check your work

- ☐ Cluster quorate with two nodes
- ☐ VM migrated live between nodes
- ☐ Table separates kernel module, emulator, and management layer

If it goes wrong

'KVM virtualisation configured, but not available'

The nested checkbox is off for the Proxmox VM in Workstation.

Two-node cluster loses quorum when one node is down

Expected. Production clusters use three nodes or a QDevice.

CLEAN UP Power off pve1 and pve2.

Deliver a virtual desktop through a broker

Host a desktop as a VM, broker access through a web gateway, and create a small login storm.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■ ■ ■ Intermediate	6 GB		Labs 12, 23
PLATFORM Ubuntu Desktop VM + Apache Guacamole				

You will

- Publish a desktop over RDP
- Use a connection broker
- Observe host impact of simultaneous logins

Before you start

- An Ubuntu Desktop VM (2 vCPU, 3 GB) named desk01, sealed as in Lab 12
- ctr01 with Docker (Lab 22), or install Docker now

Procedure

- 1 In desk01, install and enable xrdp.

```
guest$ sudo apt install -y xrdp && sudo systemctl enable --now xrdp
guest$ ss -tlnp | grep 3389
```

EXPECTED OUTPUT

```
LISTEN 0 2 *:3389 *: users:(("xrdp",pid=1220,fd=11))
```

- 2 Connect from your laptop with Remote Desktop Connection. Note scrolling, video playback, clipboard, and sound. Log out.
- 3 On ctr01, run Guacamole as a broker (guacd plus the web app). Follow the project's Docker instructions to initialize its database, then open <http://<ctr01>:8080/guacamole>.

CHECKPOINT You can log in to the Guacamole web page.

- 4 Add desk01 as an RDP connection in Guacamole and connect from a browser. Compare latency and quality with direct RDP.
- 5 Make two linked clones of desk01 and log in to all three within 30 seconds. Watch your laptop's CPU and disk in Task Manager.

EXPECTED OUTPUT

```
Disk 0 (C:) Active time 100% Response time 42 ms # the login storm
```

- 6 Write whether a 20-person call center should get persistent or non-persistent desktops, and why.

Check your work

- ☐ Desktop reachable directly and through the broker
- ☐ Login storm visible as a CPU or disk spike
- ☐ Persistent vs non-persistent choice justified

If it goes wrong

Black screen after RDP login	The user is already logged in on the console. Log out there, or use a separate RDP-only user.
-------------------------------------	---

Laggy through Guacamole	Lower the colour depth in the connection settings.
--------------------------------	--

CLEAN UP Delete the two clones. Stop the Guacamole containers if you do not need them.

Containers and the Cloud

The next abstraction layer, how it sits on top of VMs, and where it all runs at scale.

LAB 22

Chapter 22 · From Virtual Machines to Containers

First containers, measured against a VM

Run your first containers and measure their startup time and footprint against a VM.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
60 min	 Foundation	3 GB		Lab 12
PLATFORM	Ubuntu VM in Workstation			

You will

- Install Docker Engine
- Compare container and VM startup and memory
- See namespaces and cgroups directly

Before you start

- A linked clone of golden named ctr01 (2 vCPU, 3 GB)

Procedure

- 1 Install Docker Engine.

```
guest$ curl -fsSL https://get.docker.com | sudo sh
guest$ sudo usermod -aG docker $USER && newgrp docker
guest$ docker version --format '{{.Server.Version}}'
```

- 2 Time a container from start to exit.

```
guest$ docker pull alpine
guest$ time docker run --rm alpine true
```

```
EXPECTED OUTPUT
real    0m0.612s
```

- 3 Time a VM from power-on until VMware Tools reports it running.

```
PS> Measure-Command { vmrun -T ws start C:\Lab\app01\app01.vmx nogui; do { Start-Sleep 1 }
until ((vmrun -T ws checkToolsState C:\Lab\app01\app01.vmx) -eq "running") } | Select
TotalSeconds
```

EXPECTED OUTPUT

```
TotalSeconds
-----
27.84
```

CHECKPOINT The container starts roughly 40× faster than the VM.

- 4 Run ten nginx containers and compare their total memory with one idle VM (Workstation's VM → Settings shows guest memory; Task Manager shows vmware-vmx.exe).

```
guest$ for i in $(seq 1 10); do docker run -d --name n$i nginx:alpine; done
guest$ docker stats --no-stream --format 'table {{.Name}}\t{{.MemUsage}}'
```

EXPECTED OUTPUT

```
NAME      MEM USAGE / LIMIT
n1        3.9MiB / 2.8GiB
n2        3.8MiB / 2.8GiB
...
```

- 5 Prove every container shares ctr01's kernel.

```
guest$ uname -r; docker run --rm alpine uname -r
guest$ sudo lsns -t pid | head -5
```

- 6 Create a process namespace by hand, without Docker.

```
guest$ sudo unshare --pid --fork --mount-proc bash -c 'ps aux'
```

EXPECTED OUTPUT

```
USER      PID %CPU %MEM COMMAND
root      1  0.0  0.1 bash -c ps aux
root      2  0.0  0.1 ps aux      # a new PID 1: the process thinks it is alone
```

Check your work

- ☐ Container start well under a second; VM start tens of seconds
- ☐ Ten containers use less memory than one VM
- ☐ Same kernel version inside and outside the container

If it goes wrong

permission denied on docker.sock

You have not re-logged in since joining the docker group; run newgrp docker.

```
CLEAN UP  docker rm -f $(docker ps -aq). Keep ctr01.
```

Build, persist, and publish an image

Build a custom image, keep its data alive across container replacement with a volume, and push it to a private registry.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
90 min	■ ■ ■ Intermediate	3 GB		Lab 22
PLATFORM	ctr01			

You will

- Write a Dockerfile and read its layers
- Separate ephemeral and persistent storage
- Run and push to a registry

Before you start

- ctr01 with Docker

Procedure

- 1 Create a working directory with a page and a Dockerfile.

```
guest$ mkdir labapp && cd labapp
guest$ echo "<h1>lab image v1</h1>" > index.html
guest$ cat > Dockerfile <<'EOF'
FROM nginx:1.27-alpine
COPY index.html /usr/share/nginx/html/
VOLUME /data
EXPOSE 80
EOF
```

- 2 Build and inspect the layers.

```
guest$ docker build -t labapp:1.0 .
guest$ docker history labapp:1.0 --format 'table {{.CreatedBy}}\t{{.Size}}' | head -5
```

```
EXPECTED OUTPUT
CREATED BY                               SIZE
EXPOSE map[80/tcp:{}]                    0B
VOLUME [/data]                           0B
COPY index.html /usr/share/nginx/html/ # bui... 25B
CMD ["nginx" "-g" "daemon off;"]         0B
```

- 3 Prove the container filesystem is ephemeral.

```
guest$ docker run -d --name app -p 8080:80 labapp:1.0
guest$ docker exec app sh -c 'echo hi > /tmp/note'
guest$ docker rm -f app && docker run -d --name app -p 8080:80 labapp:1.0
guest$ docker exec app cat /tmp/note
```

EXPECTED OUTPUT

```
cat: can't open '/tmp/note': No such file or directory
```

4 Repeat with a named volume.

```
guest$ docker rm -f app && docker volume create appdata
guest$ docker run -d --name app -p 8080:80 -v appdata:/data labapp:1.0
guest$ docker exec app sh -c 'date > /data/stamp'
guest$ docker rm -f app && docker run -d --name app -p 8080:80 -v appdata:/data labapp:1.0
guest$ docker exec app cat /data/stamp
```

CHECKPOINT The timestamp survives the container being replaced.

5 Run a private registry and push.

```
guest$ docker run -d -p 5000:5000 --restart=always --name registry registry:2
guest$ docker tag labapp:1.0 localhost:5000/labapp:1.0
guest$ docker push localhost:5000/labapp:1.0
guest$ curl -s localhost:5000/v2/_catalog
```

EXPECTED OUTPUT

```
{"repositories":["labapp"]}
```

6 Change index.html, rebuild as 1.1, and note which build steps say CACHED.

Check your work

- ☐ curl localhost:8080 shows your page
- ☐ Data in the volume survived; data outside it did not
- ☐ Registry catalog lists labapp

If it goes wrong

Container exits immediately

docker logs <name> shows why the main process ended.

Port already allocated

A previous container still holds it; docker ps -a and remove it.

CLEAN UP Keep the registry and labapp images for Labs 24 and 27.

A small cluster that survives a node failure

Build a three-node Kubernetes cluster, deploy and scale an app, and watch it self-heal when a node dies.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2–3 hrs	■■■ Advanced	5 GB	(2 + 1.5 + 1.5)	Labs 12, 23
PLATFORM	k3s on three Workstation VMs			

You will

- Install a control plane and two workers
- Use deployments and services
- Observe rescheduling after node loss

Before you start

- Three linked clones of golden: k8s-cp (2 vCPU, 2 GB), k8s-w1 and k8s-w2 (1 vCPU, 1.5 GB)
- Power off everything else

Procedure

- 1 Install the server on k8s-cp and read the join token.

```
cp$ curl -sfl https://get.k3s.io | sh -
cp$ sudo cat /var/lib/rancher/k3s/server/node-token
```

- 2 Join both workers.

```
w$ curl -sfl https://get.k3s.io | K3S_URL=https://<cp-ip>:6443 K3S_TOKEN=<token> sh -
cp$ sudo kubectl get nodes
```

EXPECTED OUTPUT				
NAME	STATUS	ROLES	AGE	VERSION
k8s-cp	Ready	control-plane,master	6m	v1.33.x+k3s1
k8s-w1	Ready	<none>	2m	v1.33.x+k3s1
k8s-w2	Ready	<none>	1m	v1.33.x+k3s1

CHECKPOINT Three nodes Ready.

- 3 Deploy an app and expose it.

```
cp$ sudo kubectl create deployment web --image=nginx:alpine --replicas=3
cp$ sudo kubectl expose deployment web --port=80 --type=NodePort
cp$ sudo kubectl get pods,svc -o wide
```

- 4 Scale up and roll out an update.

```
cp$ sudo kubectl scale deployment web --replicas=6
cp$ sudo kubectl set image deployment/web nginx=nginx:1.27-alpine
cp$ sudo kubectl rollout status deployment/web
```

EXPECTED OUTPUT

deployment "web" successfully rolled out

- 5 Delete one pod and time how fast it is replaced.

```
cp$ sudo kubectl delete pod $(sudo kubectl get pod -l app=web -o name | head -1)
cp$ sudo kubectl get pods -l app=web
```

- 6 Power off k8s-w1 in Workstation (hard). Watch the node go NotReady and, after about five minutes, its pods get rescheduled.

```
cp$ sudo kubectl get nodes -w
cp$ sudo kubectl get pods -o wide -w
```

EXPECTED OUTPUT

```
k8s-w1    NotReady    <none>    40m    v1.33.x+k3s1
web-6d4b...-x7k2p    1/1    Terminating    0    12m    10.42.1.8    k8s-w1
web-6d4b...-m9q4r    1/1    Running         0    4s    10.42.2.11    k8s-w2
```

Check your work

- ☐ Three nodes Ready
- ☐ Six replicas after scaling; rollout completed
- ☐ After k8s-w1 loss, all replicas run on remaining nodes

If it goes wrong

Pod stuck Pending	kubectl describe pod <name>: usually insufficient CPU or memory on remaining nodes.
CrashLoopBackOff	kubectl logs <pod> --previous shows why the last run died.
Service has no endpoints	The service selector does not match pod labels; compare kubectl get endpoints web with pod labels.

CLEAN UP Power k8s-w1 back on. Keep the cluster for Lab 25.

Trace one workload through every layer

Follow a pod down to a container, a VM, and a process on your laptop, then inject a fault at the bottom and see it surface at the top.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
90 min	■■■ Advanced	5 GB		Lab 24
PLATFORM	k3s cluster on Workstation			

You will

- Map pod → node → VM → host process
- Find the same workload in each layer's tools
- Attribute a pod symptom to a host cause

Before you start

- Cluster from Lab 24

Procedure

- 1 Pick one web pod and find its node.

```
cp$ sudo kubectl get pod -l app=web -o wide
```

- 2 On that node, find the container and its Linux process.

```
w$ sudo k3s crictl ps --name nginx
w$ sudo k3s crictl inspect <container-id> | grep -m1 '"pid"'
w$ ps -o pid,cmd -p <pid>
```

EXPECTED OUTPUT

```
"pid": 2841,
PID CMD
2841 nginx: master process nginx -g daemon off;
```

- 3 On your laptop, find the Workstation process that is that node's VM.

```
PS> Get-CimInstance Win32_Process -Filter "Name='vmware-vmx.exe'" | Select ProcessId,
@{n='VM';e={$_.CommandLine -split ' ')[-2]}}
```

EXPECTED OUTPUT

```
ProcessId VM
----- --
18344 C:\Lab\k8s-w2\k8s-w2.vmx
```

CHECKPOINT You can name the pod's container PID, node VM, and laptop process.

- 4 Draw the stack: container PID → node VM → vmware-vmx.exe → Windows → physical CPU.
- 5 Measure request latency to the service, then create CPU contention on your laptop (run stress in two other VMs, or a CPU-heavy app) and measure again.

```
guest$ for i in $(seq 20); do curl -o /dev/null -s -w '%{time_total}\n' http://<node-ip>:  
<nodeport>; done
```

- 6 Using kubectl alone, try to find the cause. Then check the node's steal time and the laptop's Task Manager. Write which layer revealed it.

```
w$ vmstat 1 5
```

Check your work

- ☐ Complete stack diagram for one pod
- ☐ Latency rose under host contention
- ☐ You can state why kubectl could not see the cause

If it goes wrong

Steal time shows 0 in Workstation guests	Workstation may not report steal time. Compare guest CPU time with wall-clock time, or watch the laptop's CPU graph.
---	--

CLEAN UP Stop the contention. Power off the cluster.

A cloud VM, a managed container, and the map back

Launch a cloud instance and a managed container service, and map each setting to the on-premises concept you already built.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
90 min	■ ■ □ Intermediate	None	local	Labs 4, 23
PLATFORM	Any major cloud free tier			

You will

- Launch and inspect a cloud VM
- Deploy an image to a managed container service
- Place each on the shared-responsibility model

Before you start

- A cloud account
- labapp pushed to that cloud's container registry

Procedure

- 1 Create a billing alert at a small amount before launching anything.

CHECKPOINT The budget alert is active.

- 2 Launch the smallest free-tier Linux instance. As you click through, note which on-prem choice each setting mirrors: instance type (vCPU/RAM), image (template), volume (virtual disk), security group (firewall and microsegmentation), subnet (port group and VLAN).

- 3 SSH in and ask what it is running on.

```
cloud$ systemd-detect-virt
cloud$ lscpu | grep -i hypervisor
cloud$ lsblk
```

EXPECTED OUTPUT

```
kvm
Hypervisor vendor:    KVM
nvme0n1      259:0      0    8G  0 disk      # the virtual disk, presented as NVMe
```

- 4 Deploy labapp to a managed container service (for example Cloud Run, Azure Container Apps, or AWS App Runner) and open its URL.
- 5 Fill a three-column table: cloud feature, on-prem concept, who is responsible (you or the provider).
- 6 Revisit your Lab 1 inventory and correct any entries you now classify differently.

Check your work

- ☐ Instance reports a hypervisor
- ☐ Container URL serves your page
- ☐ Mapping table has at least eight rows with responsibility

If it goes wrong

Charges appearing

Look for unattached volumes, reserved IPs, NAT gateways, and load balancers. They bill after the instance is gone.

CLEAN UP Terminate the instance, delete its volume and any public IP, delete the container service and registry image. Check billing shows nothing running.

Security, Troubleshooting, and Scale

Hardening, the full diagnostic framework, staged faults, capacity, and your next steps.

LAB 27

Chapter 27 · Virtualization Security

Harden the management plane and scan an image

Close the realistic attack path into the hypervisor, then find the vulnerabilities and secrets frozen into your images.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
2 hrs	■ ■ ■ Intermediate	8 GB		Labs 3, 23
PLATFORM	esxi1 + ctr01 with Trivy			

You will

- Restrict and authenticate management access
- Apply least privilege
- Scan images for CVEs and secrets

Before you start

- Snapshot esxi1 in Workstation
- Console access to esxi1 in case you lock yourself out

Procedure

- 1 Create a named admin account and stop using root day to day.

```
esxi# esxcli system account add -i alex -p '<password>' -c '<password>'
esxi# esxcli system permission set -i alex -r Admin
esxi# esxcli system permission list
```

EXPECTED OUTPUT				
Principal	Is Group	Role	Role Description	
-----	-----	----	-----	
alex	false	Admin	Full access rights	
root	false	Admin	Full access rights	

- 2 Restrict SSH and the Host Client to your laptop's address only.

```
esxi# esxcli network firewall ruleset set -r sshServer --allowed-all false
esxi# esxcli network firewall ruleset allowedip add -r sshServer -i 192.168.80.1
esxi# esxcli network firewall ruleset set -r webAccess --allowed-all false
esxi# esxcli network firewall ruleset allowedip add -r webAccess -i 192.168.80.1
esxi# esxcli network firewall ruleset allowedip list | grep -E 'sshServer|webAccess'
```

- 3 From another VM on vmnet8 (e.g. web01), confirm the Host Client is no longer reachable.

```
guest$ curl -k -m 5 https://192.168.80.11/ -o /dev/null -w '%{http_code}\n'
```

EXPECTED OUTPUT

```
000      # timed out: blocked
```

CHECKPOINT Your laptop can still reach the Host Client; web01 cannot.

- 4 Set account lockout, shell timeouts, and stop SSH when not needed.

```
esxi# esxcli system settings advanced set -o /Security/AccountLockFailures -i 5
esxi# esxcli system settings advanced set -o /UserVars/ESXiShellInteractiveTimeout -i 600
esxi# vim-cmd hostsvc/disable_ssh
```

- 5 Install Trivy on ctr01 and scan your image and an old public one.

```
guest$ curl -sfl
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sudo sh -s --
-b /usr/local/bin
guest$ trivy image --severity HIGH,CRITICAL labapp:1.0
guest$ trivy image --severity HIGH,CRITICAL nginx:1.21
```

EXPECTED OUTPUT

```
nginx:1.21 (debian 11.x)
Total: 120 (HIGH: 95, CRITICAL: 25)      # counts vary with the database date
```

- 6 Plant a fake secret, then find it. Then list the secrets that could be sitting in your golden template (SSH host keys, shell history, cloud credentials).

```
guest$ mkdir s && echo "AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY" >
s/.env
guest$ trivy fs --scanners secret s/
```

Check your work

- ☐ Daily admin is a named user
- ☐ Host Client blocked from other VMs
- ☐ Old image has many more findings than current
- ☐ Planted secret detected

If it goes wrong

Locked yourself out

Use the esxi1 console in Workstation: F2 → Troubleshooting Options, or revert the snapshot.

Trivy DB download fails

The first scan needs internet access.

CLEAN UP Delete the s/ folder. Keep the hardening. Re-enable SSH only when a lab needs it.

Diagnose a multi-layer fault to root cause

Apply the complete framework to a fault whose symptom and cause live in different layers, and document every step.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
2 hrs	■■■ Advanced	11 GB	Labs 5, 9–15
PLATFORM esxi1, esxi2, stor1			

You will

- Move from symptom to layer systematically
- Correlate guest, host, and infrastructure logs
- Use substitution and isolation tests

Before you start

- Your Lab 5 plan
- tiny01 running on nas-nfs
- A fault menu prepared in advance (step 1)

Procedure

- 1 Prepare five fault scripts on your laptop, one per file: (a) disconnect esxi1's storage NIC; (b) fill nas-nfs to 99% from stor1 with fallocate; (c) on stor1, throttle the NFS export with tc qdisc netem delay 80ms; (d) leave a snapshot on tiny01 and write 2 GB in the guest; (e) set a 300 MHz CPU limit on tiny01 with PowerCLI.

- 2 Pick one without looking.

```
PS> Get-ChildItem C:\Lab\faults\*.ps1 | Get-Random | ForEach-Object { & $_.FullName *> $null }
```

- 3 Define: write the symptom as a user would report it, then as a measurable statement.
- 4 Blast radius: check every VM on esxi1 and esxi2. Is it one VM, one host, or everything on nas-nfs? Write what that implies.

CHECKPOINT You have named the suspect layer before looking at any logs.

- 5 Gather evidence from each layer and correlate timestamps.

```
guest$ journalctl -p warning --since "-30 min"
esxi# grep -iE 'nfs|apd|lost|latency' /var/log/vmkernel.log | tail -20
esxi# esxtop      # v: LAT/rd  c: %RDY, %MLMTD
stor1$ df -h /srv/vmstore; tc qdisc show
```

- 6 Isolate: move tiny01 to the other host or to local storage. If the symptom follows the VM, the cause is in the VM or its config; if it stays behind, it is in the infrastructure.
- 7 Make one change, evaluate, and write the root cause in one sentence. Compare with your Lab 5 plan: what did you do differently?

Check your work

- ☐ Root cause matches the injected fault
- ☐ Every step recorded with evidence
- ☐ Only one change made before the fix was confirmed

If it goes wrong

Evidence contradicts your hypothesis	Good. Drop the hypothesis, not the evidence.
--------------------------------------	--

CLEAN UP Reverse the injected fault. Keep the fault scripts for Labs 29–30.

Attribute four staged slowdowns

For each staged slowdown, name the constrained resource and the responsible layer using measurements, not guesses.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
2 hrs	■■■ Advanced	5 GB	Labs 6–8, 13
PLATFORM	esxi1		

You will

- Turn 'it's slow' into a metric
- Run the four-resource investigation
- Find the noisy neighbor

Before you start

- tiny01 as the victim, tiny02–04 as neighbors, all on esxi1
- fio, stress-ng, and iperf3 in each

Procedure

- 1 Baseline the victim and write down the numbers.

```
victim$ stress-ng --cpu 1 --timeout 30s --metrics-brief
victim$ sudo fio --name=lat --filename=/tmp/f --size=256M --rw=randread --bs=4k --iodepth=1
--direct=1 --runtime=30 --time_based
```

EXPECTED OUTPUT

```
cpu  bogo ops/s  402.1
lat: read: IOPS=2860, clat avg=348us
```

- 2 Scenario A, CPU: overcommit with stress-ng --cpu in all neighbors. Find it in esxtop c (%RDY on the victim).
- 3 Scenario B, Memory: push esxi1 into swap as in Lab 7. Find it in esxtop m (state, MCTLSZ, SWCUR).
- 4 Scenario C, Storage: run a heavy fio write in one neighbor. Find it in esxtop v: which VM has the highest CMDS/s, and did the victim's LAT/rd rise?

```
neighbor$ sudo fio --name=noisy --filename=/tmp/n --size=1G --rw=randwrite --bs=64k --
iodepth=32 --ioengine=libaio --direct=1 --runtime=300 --time_based
```

EXPECTED OUTPUT

GID	VMNAME	VDEVNAME	NVDISK	CMDS/s	READS/s	WRITES/s	LAT/rd	LAT/wr
2245	tiny02	-	1	4210.3	1.2	4209.1	9.21	31.40
2210	tiny01	-	1	104.5	104.5	0.0	18.77	0.00

CHECKPOINT You can name the noisy neighbor from esxtop alone.

- 5 Scenario D, Network: run iperf3 between two neighbors. Find the busy vmnic in esxtop n.

```
nbr-a$ iperf3 -s
nbr-b$ iperf3 -c <nbr-a> -t 120 -P 4
```

- 6 For each scenario, fill the row: symptom in victim, resource, layer (guest, host, or infrastructure), metric and value, culprit VM.

Check your work

- ☐ Four rows completed
- ☐ Each culprit identified by metric
- ☐ Guest-only causes distinguished from host-level causes

If it goes wrong

Storage and CPU symptoms look alike in the guest	High iowait (wa) in vmstat points to storage; high %RDY on the host points to CPU.
Network scenario barely affects the victim	VM-to-VM traffic on the same vSwitch never leaves the host. Put nbr-b on esxi2 to cross the uplink.

CLEAN UP Stop all load generators.

Recover from five staged failures

Diagnose and recover a sequence of hard failures, choosing repair, restore, or rebuild for each.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
3 hrs	■■■ Advanced	11 GB	Labs 8, 12, 14–16
PLATFORM esxi1, esxi2, stor1			

You will

- Separate power-on, boot, and runtime failures
- Recover storage and snapshot problems
- Make and justify the recovery decision

Before you start

- A current backup of tiny01 (Lab 16)
- Snapshots of esxi1 and esxi2 in Workstation

Procedure

- 1 Won't power on: set tiny01's memory above what esxi1 can provide and power on. Read the error and fix it.

EXPECTED OUTPUT

Failed to power on virtual machine tiny01. Insufficient memory resources.

- 2 Won't boot: remove tiny01's disk from the VM (keep the file) and power on. Separate a virtual-hardware fault from a guest-OS fault by what the console shows. Re-attach the existing disk.

EXPECTED OUTPUT

```
Attempting to start up from:
→ EFI VMware Virtual SCSI Hard Drive (0.0)... unsuccessful.
→ EFI Network...
```

- 3 Full datastore: fill nas-nfs from stor1 while tiny01 writes. Recover without data loss.

```
stor1$ sudo fallocate -l 50G /srv/vmstore/filler
stor1$ sudo rm /srv/vmstore/filler      # the fix
esxi# vim-cmd vmsvc/message <vmid> _vmx1 1
```

CHECKPOINT tiny01 resumes with its data intact.

- 4 Host failure: power off esxi1 hard with tiny01 running. Decide: wait for the host, restore from backup elsewhere, or register on esxi2 (Lab 15). Do it and time it.
- 5 Network isolation: move tiny01 to a port group with VLAN 99 (nothing else there). Diagnose from the guest outward: link, IP, gateway, port group, VLAN.

- 6 For each failure, record detection, root cause, decision (repair, restore, or rebuild), and time to recover.

Check your work

- ☐ All five recovered
- ☐ Each decision justified by time and data-loss cost
- ☐ No VM left in a broken state

If it goes wrong

Registering on esxi2 fails with a lock error esxi1's lock has not expired. Wait a minute (Lab 15).

CLEAN UP Confirm every VM boots and every datastore is accessible: esxcli storage filesystem list.

Size the host and reclaim the waste

Measure the real capacity of esxi1, compute a safe consolidation ratio, and identify oversized VMs.

TIME	LEVEL	RAM	POWERED ON	BUILDS ON
90 min	Intermediate	5 GB		Labs 13, 17
PLATFORM	esxi1			

You will

- Collect per-VM allocation and usage
- Find the limiting resource
- Compute N+1 capacity

Before you start

- tiny01–04 running a light mixed workload for at least 30 minutes

Procedure

- 1 Capture 10 minutes of performance data in batch mode.

```
esxi# esxtop -b -d 10 -n 60 > /vmfs/volumes/datastore1/perf.csv
PS> scp root@192.168.80.11:/vmfs/volumes/datastore1/perf.csv .
```

- 2 Export allocation and current usage per VM.

```
PS> Get-VM | Select Name, NumCpu, MemoryGB, @{n='CPUUsageMHz';e=
{$_ .ExtensionData.Summary.QuickStats.OverallCpuUsage}}, @{n='ActiveMemMB';e=
{$_ .ExtensionData.Summary.QuickStats.GuestMemoryUsage}}
```

EXPECTED OUTPUT				
Name	NumCpu	MemoryGB	CPUUsageMHz	ActiveMemMB
tiny01	2	1.50	88	143
tiny02	1	0.75	412	501
tiny03	2	1.50	21	92
tiny04	1	0.75	35	180

- 3 For each resource, divide total VM peak usage by host capacity. The highest ratio is your limiting factor.

CHECKPOINT You can name one limiting resource with a number.

- 4 Compute a safe consolidation ratio. Use targets such as: vCPU:pCPU $\leq$ 4:1, memory committed $\leq$ 90% of host after failover, datastore $\leq$ 80% full. For a two-host setup (esxi1 + esxi2), check N+1: can one host carry every VM if the other fails?
- 5 Flag oversized VMs: peak CPU below 20% of allocation, or active memory below 50% of allocation. Right-size one and record the capacity freed.
- 6 Write a sprawl rule: every VM gets an owner and an expiry note, and anything idle for 30 days is reviewed.

Check your work

- ☐ Limiting resource identified with numbers
- ☐ N+1 result shown with the math
- ☐ At least one VM right-sized

If it goes wrong

Usage looks tiny everywhere

A single reading shows the current moment, not the peak. Use the batch capture and take the 95th percentile.

CLEAN UP Delete perf.csv from the datastore.

Assess yourself and write a 90-day plan

Rate your skills against every lab, choose a specialization, and commit to a dated 90-day plan with a running lab.

TIME	LEVEL	RAM POWERED ON	BUILDS ON
60 min	■□□ Foundation	None	All labs
PLATFORM	Paper or text editor		

You will

- Self-assess across the six parts
- Choose a specialization
- Write a dated, measurable plan

Before you start

- Your notes from all previous labs

Procedure

- 1 For each of the 31 labs, rate yourself 1–4: 1 = followed the steps, 2 = could repeat it unaided, 3 = could troubleshoot it, 4 = could teach it.

- 2 Average by part. Your lowest part is your first 30 days.

CHECKPOINT Every lab has a rating.

- 3 Pick one specialization: enterprise virtualization, cloud, containers and Kubernetes, or virtual desktops. Write two sentences on why.
- 4 Choose one certification or vendor track that matches it and list the exam objectives these labs already covered.
- 5 Write the plan in three 30-day blocks. Each block has something you can demonstrate, a reading goal, and a date.
- 6 Decide which lab VMs stay permanently and set a recurring reminder to patch them. Note your ESXi evaluation end date and your plan for it (free license, rebuild, or Proxmox).

Check your work

- ☐ Every lab rated
- ☐ Specialization chosen and justified
- ☐ Three dated blocks, each with a demonstrable outcome

If it goes wrong

Everything rates 1

Redo the Part II labs without the guide. That is where fluency starts.

CLEAN UP Archive your lab notes with the plan.

Lab setup and prerequisites

Do this once, before Lab 3. Allow two to three hours including downloads.

A1 • Laptop requirements

Item	Minimum	Notes
Memory	16 GB	About 11–12 GB is usable for VMs once Windows and Workstation are running
CPU	4 cores / 8 threads	Intel VT-x with EPT, or AMD-V with RVI, enabled in firmware (Lab 2)
Storage	250 GB free SSD	A spinning disk makes nested ESXi painfully slow
OS	Windows 10/11 64-bit	A Linux host works too; swap PowerShell steps for their shell equivalents
Network	Internet access	Needed for downloads, package installs, Hands-on Labs, and Lab 26

With 32 GB or more, you can run every lab locally, including a nested vCenter, and skip the Hands-on Labs sections.

A2 • Free the CPU from Windows' own hypervisor

Workstation can only pass virtualization through to nested hypervisors when Windows is not running Hyper-V underneath it. Turn off the Hyper-V family of features and Memory Integrity, then reboot.

```
PS> Disable-WindowsOptionalFeature -Online -NoRestart -FeatureName Microsoft-Hyper-V-All,
VirtualMachinePlatform, HypervisorPlatform
PS> bcdedit /set hypervisorlaunchtype off
# Settings → Privacy & security → Windows Security → Device security → Core isolation → Memory
integrity: Off
PS> Restart-Computer
```

After the reboot, re-run the VBS check from Lab 1. It must return 0.

TRADE-OFF WSL2 and Docker Desktop stop working while this is off. To switch back later, run `bcdedit /set hypervisorlaunchtype auto`, re-enable the features, and reboot.

A3 • Downloads

Software	Where	First used
VMware Workstation Pro	Broadcom support portal (free account). Free for personal use.	Lab 3
VMware ESXi 8.0 Update 3e or later (vSphere Hypervisor)	Broadcom support portal → Free Downloads → VMware vSphere Hypervisor. Note the free license key on that page.	Lab 3
Ubuntu Server 24.04 LTS and Ubuntu Desktop 24.04 LTS	ubuntu.com/download	Labs 4, 21
PowerShell 7 and VCF PowerCLI	Microsoft Store or GitHub; PowerCLI from the PowerShell Gallery	Lab 17
Windows Server 2025 evaluation	Microsoft Evaluation Center (180-day evaluation)	Lab 19
Proxmox VE	proxmox.com/downloads	Lab 20
VMware Hands-on Labs	labs.hol.vmware.com. Free, runs in a browser, uses the same Broadcom account.	Lab 11

A4 • The ESXi 60-day clock

A fresh ESXi install runs in 60-day evaluation mode with every feature and a full API. The free license, once applied, makes the API read-only and blocks vCenter. Plan to finish **Labs 3–18 within 60 days of installing esxi1**. Labs 16 and 17 need the write API. If the clock runs out, rebuilding the nested host from the ISO starts a new evaluation.

A5 • Virtual networks

Open Workstation → Edit → Virtual Network Editor → Change Settings. Configure these four networks.

Network	Type	Subnet	Purpose
vmnet8	NAT, DHCP on	192.168.80.0/24	Management and VM traffic. Gateway .2, DHCP range .128–.254
vmnet1	Host-only	default	Lab 10 connection-mode tests
vmnet2	Host-only, DHCP off	10.30.0.0/24	Storage (NFS, iSCSI)
vmnet3	Host-only, DHCP off	10.20.0.0/24	vMotion / migration

Fixed addresses on vmnet8: laptop .1, gateway .2, stor1 .10, esxi1 .11, esxi2 .12, pve1 .21, pve2 .22. The storage and migration networks reuse the same last number (esxi1 is 10.30.0.11 and 10.20.0.11).

A6 • Machines you will build

Name	Runs on	Size	First used
esxi1	Workstation	4 vCPU · 8 GB to install, then 5 GB · 40 + 80 GB	Lab 3
web01 · golden	Workstation	2 vCPU · 2 GB · 20 GB	Labs 4, 12
tiny01–04	esxi1	1–2 vCPU · 768 MB–1.5 GB · 8 GB thin	Lab 6
esxi2 · stor1	Workstation	5 GB · 1 GB with a 60 GB data disk	Lab 9
hv1 · hv2	Workstation	2 vCPU · 4.5 GB · 60 GB each	Lab 19
pve1 · pve2	Workstation	2 vCPU · 3.5 GB · 32 GB each	Lab 20
desk01	Workstation	2 vCPU · 3 GB	Lab 21
ctr01	Workstation	2 vCPU · 3 GB	Lab 22
k8s-cp · w1 · w2	Workstation	2 GB · 1.5 GB · 1.5 GB	Lab 24

A7 · What to power on, by part

Labs	Powered on	VM RAM
3–8 · 13 · 16–18 · 29 · 31	esxi1 (with its tiny VMs inside)	5 GB
4 · 10 · 12	web01 and its clones	2–4 GB
9 · 11 · 14 · 15 · 28 · 30	esxi1, esxi2, stor1. Close every other app.	11 GB
19	hv1, hv2	9 GB
20	pve1, pve2	7 GB
21–25 · 27	ctr01, desk01, or the k3s nodes (plus esxi1 for Lab 27)	3–8 GB

A8 · When one laptop is not enough

Labs 11, 14, 15, and 18 need vCenter, which will not fit alongside two ESXi hosts in 16 GB. Each of those labs does the underlying mechanics by hand on esxi1 and esxi2, then continues in a VMware Hands-on Lab. Start the Hands-on Lab before the local steps; it can take several minutes to provision.

If two ESXi hosts will not fit at all, pair with another learner (one runs esxi1, the other runs esxi2), or lower both hosts to 4.5 GB and accept slower guests.

THE SNAPSHOT RULE

1. Snapshot in Workstation before any step that changes a host's configuration. Name it for what it protects.
2. Delete snapshots when the lab ends. Lab 12 shows why forgotten snapshots are dangerous.
3. Never run these labs on a network you depend on for work. Use NAT when in doubt.

Expected results and instructor notes

Check your results here after each lab. Instructor notes are for anyone running this guide with a group or a study partner; self-paced learners will also find the common pitfalls useful.

LAB 01 Find the virtualization already in your life

EXPECTED RESULTS

- A typical Windows 11 laptop reports 'A hypervisor has been detected' because VBS, Memory Integrity, or WSL2 runs Windows itself on Hyper-V.
- Email, streaming, and banking: server virtualization. Cloud file sync: storage virtualization. Corporate VPN or SD-WAN: network virtualization. A browser-based desktop: desktop virtualization. A streamed app or WSL app: application virtualization.
- A good worked example names the abstraction (hardware separated from OS), the benefit (consolidation, portability, fast provisioning), and a cost (overhead, shared-host risk, noisy neighbors).

INSTRUCTOR NOTES

- Use this as an icebreaker. Most learners are surprised their own laptop already runs a hypervisor.
- Accept 'server' for any SaaS entry. The goal is noticing layers, not perfect classification.
- Flag learners with VBS status 2 now. They will hit the Workstation nested-VT error in Lab 3 otherwise.

LAB 02 Confirm your CPU can virtualize

EXPECTED RESULTS

- Any Intel Core or AMD Ryzen CPU from the last decade supports VT-x/AMD-V and EPT/RVI.
- SLAT matters because without it the hypervisor must maintain shadow page tables in software, which is the largest single source of memory-virtualization overhead.

INSTRUCTOR NOTES

- Budget ten minutes of the session for firmware reboots. Some vendors ship virtualization disabled.
- Ask: why did CPU vendors add these features at all? Look for the answer 'trapping privileged instructions in software was too slow'.

LAB 03 Stand up Workstation and a nested ESXi host

EXPECTED RESULTS

- HV Support values: 0 = not supported, 1 = supported but not enabled, 2 = enabled but not usable, 3 = enabled and usable.
- You now have both types: Workstation is Type 2 (hosted on Windows), ESXi is Type 1 (bare metal, even though its 'metal' is virtual).

INSTRUCTOR NOTES

- The Hyper-V conflict causes most failures here. Have learners run Appendix A's check before class.
- Remind learners the free ESXi install runs in 60-day evaluation mode. Advise finishing the ESXi-heavy labs (3–18) inside that window.
- Ask: what can this lab not teach you that a physical server would? Good answers mention real NIC and storage hardware behavior.

LAB 04 Build a VM deliberately, setting by setting

EXPECTED RESULTS

- Shutdown asks the guest OS to stop cleanly. Suspend freezes vCPUs and saves RAM to a .vmss file so it can resume. Hard stop cuts power: no guest cleanup, possible filesystem repair.
- Over-allocating vCPUs or RAM wastes host resources and can make scheduling worse (Lab 6). Under-allocating causes guest swapping and slow installs.
- UEFI is required for Secure Boot and modern OSes and for disks over 2 TB. BIOS remains for legacy guests.

INSTRUCTOR NOTES

- Insist learners write choices before clicking. That is the point of the lab.
- A common confusion: suspend in Workstation is not the same as sleep inside the guest. Show the .vmss file appear.

LAB 05 Write a diagnostic plan before touching anything

EXPECTED RESULTS

- Strongest first hypothesis: the backup job left a snapshot on fin-app01 (or several VMs) or is still consolidating, raising datastore latency.
- Blast-radius check: compare latency across all VMs on the host (esxtop disk view, DAVG/GAVG per VM). If all are slow, the shared storage layer is first. If only fin-app01, check its snapshots and guest.
- Evidence per layer. Guest: OS logs and app logs. VM: snapshot count and delta size in the datastore browser, vmware.log. Host: esxtop CPU %RDY, memory MCTL/SWCUR, vmkernel.log. Storage: DAVG in esxtop, array latency.
- One change: consolidate or delete the leftover snapshot, then compare latency with the pre-09:00 baseline.

INSTRUCTOR NOTES

- Grade the reasoning, not the answer. Accept any first layer that is justified by blast radius.
- Push back on plans that change more than one thing at once.

LAB 06 Overcommit CPU and watch ready time appear

EXPECTED RESULTS

- Expect baseline %RDY under 5%. Under 2:1 contention, expect 40–60% %RDY per vCPU and roughly half the baseline bogo ops/s.
- Per-vCPU guidance: sustained %RDY above ~5% per vCPU is noticeable, above ~10% is a problem.
- The 4-vCPU VM is slower per vCPU because the scheduler must find more free cores at once and keep its vCPUs progressing together (co-scheduling, shown as %CSTP).
- Overcommit usually works because most VMs are idle most of the time. It fails when many VMs are busy at once.

INSTRUCTOR NOTES

- Nested esxtop numbers are noisier than physical ones. Focus learners on the change, not the absolute value.
- If laptops struggle, run with 3 VMs instead of 4 and 2 vCPUs each.

LAB 07 Drive a host from ballooning into swap

EXPECTED RESULTS

- Expected order: page sharing (while idle) → ballooning (soft state) → compression → host swapping (hard/low state).
- Host swapping is where performance dies because the guest does not know its 'RAM' is now on disk; every touch of a swapped page is a disk read.
- Memory overcommit is more dangerous than CPU overcommit: CPU contention makes things wait, memory contention makes them page to disk or crash.
- ShareForceSalting = 2 (default) limits sharing to within a VM to prevent cross-VM side-channel attacks.

INSTRUCTOR NOTES

- Laptop SSD swap is fast, so the collapse may be milder than on real hosts. Point that out.
- Have learners screenshot esxtop at each state for their notes.

LAB 08 Fill a thin-provisioned datastore on purpose

EXPECTED RESULTS

- ls shows provisioned size (2 GB each). du shows allocated blocks: thick = 2 GB, thin ≈ 0.
- ESXi pauses (stuns) the VM rather than corrupting its disk. The guest sees a frozen clock, not an I/O error, until you answer.
- Thin provisioning trades safety for space: it is safe only with datastore free-space alarms and a plan to add capacity.

INSTRUCTOR NOTES

- The out-of-space question is the most memorable moment in Part II. Let learners sit with it before answering.
- Discuss what would happen on a production datastore with 40 VMs if nobody watched free space.

LAB 09 Connect two hosts to shared storage

EXPECTED RESULTS

- NFS: the storage server owns the filesystem; ESXi sees files. iSCSI: ESXi gets raw blocks and formats them VMFS itself.
- NAS = file-level access over the network (NFS/SMB). SAN = block-level access (iSCSI, Fibre Channel).
- During path loss, ESXi retries I/O rather than failing it. The guest freezes; it does not crash unless the outage exceeds timeouts. Multipathing (a second vmk on a second network) prevents this.

INSTRUCTOR NOTES

- This is the RAM crunch lab. Tell learners to close browsers and chat apps.
- targetcli is the slowest step for newcomers. Consider giving the exact commands on a handout.

LAB 10 Wire VMs in bridged, NAT, and host-only modes

EXPECTED RESULTS

- Bridged: reaches everything; appears as its own device on your home network. NAT: reaches laptop, internet, and home network outbound; not reachable from other home devices. Host-only: reaches the laptop and other host-only VMs; no internet.
- Same subnet, different VLAN IDs: no connectivity, because the virtual switch keeps tagged frames separate.
- The guest sees a normal Ethernet adapter in every case; the mode is decided entirely by what the virtual switch is connected to.

INSTRUCTOR NOTES

- The prediction step is the lab. Collect a few predictions before anyone tests.
- Warn learners on campus or corporate Wi-Fi that bridged mode may be blocked.

LAB 11 Separate traffic types across two hosts

EXPECTED RESULTS

- Traffic types: management (vmk0), storage (vmk1), vMotion (vmk2), VM traffic (port groups on vSwitch0).
- Drift breaks migration and HA because the target host lacks a network with the exact name the VM expects.
- A distributed switch defines port groups, VLANs, teaming, and NIOC once in vCenter and pushes them to every host.

INSTRUCTOR NOTES

- Pair learners without enough RAM for two ESXi hosts. One runs esxi1, the other esxi2, sharing screens.
- Book Hands-on Labs slots ahead if running as a class.

LAB 12 Snapshot chains, a golden template, and custom clones

EXPECTED RESULTS

- A snapshot is not a copy: it freezes the base disk and sends new writes to a delta disk. Reverting discards the delta. Deleting merges it back.
- Snapshots are not backups: they live on the same storage as the VM and depend on the base disk.
- A linked clone shares the parent's disk read-only and stores only its differences. It is small and fast to create but breaks if the parent snapshot is deleted.
- Customization on deployment (cloud-init, or Sysprep for Windows) is what makes clones distinct machines.

INSTRUCTOR NOTES

- Show the delta files growing live while a guest writes. It makes the 'forgotten snapshot' danger concrete.
- The PowerShell backtick-quote in the guestinfo line trips people up; have the line ready to paste.

LAB 13 Shares, limits, reservations, and finding the bottleneck

EXPECTED RESULTS

- Shares are relative priority that only apply under contention. Limits are a hard ceiling, even when capacity is free. Reservations are a guarantee.
- A memory reservation shrinks the .vswp file because reserved memory never needs to be swapped: vswp size = configured – reserved.
- Tells for the hidden load. CPU: high %RDY or %USED. Memory: MCTLSZ or SWCUR. Disk: high LAT or DAVG in v/u views. Network: MbTX/s or MbRX/s near the link rate in n.

INSTRUCTOR NOTES

- Limits are the most misused control in production. Ask learners for a real reason to set one (license caps, test throttling).
- If time is short, skip the reservation step.

LAB 14 Migrate cold locally, live in the Hands-on Lab, then break it

EXPECTED RESULTS

- vMotion copies memory while the VM runs, re-copies pages that changed, then pauses for a fraction of a second to copy the last pages and CPU state, and resumes on the target.
- Requirements: both hosts see the VM's storage (or you migrate storage too), a vMotion network, compatible CPUs, the same port group names, and enough free resources on the target.
- Error mapping. Local ISO: storage/device not accessible on target. Missing port group: network. Different CPU generation: CPU compatibility (fix with EVC).

INSTRUCTOR NOTES

- The local cold-migration step shows exactly what HA will automate in Lab 15. Call that out.
- Hands-on Lab environments vary by release. Give learners the goal, not the click path.

LAB 15 Pull the plug on a host

EXPECTED RESULTS

- NFS and VMFS lock a running VM's files. The lock must expire or be released before another host can start it. This is what stops two hosts running one VM.
- HA uses network heartbeats plus datastore heartbeats. If a host misses network heartbeats but still updates datastore heartbeats, it is isolated, not failed, and HA applies the isolation response.
- Admission control reserves enough capacity to restart VMs after a host failure. Without it, the failover may not fit.
- Fault tolerance keeps a live shadow copy running on another host for zero-downtime failover; HA restarts the VM, so there is a reboot.

INSTRUCTOR NOTES

- Manual recovery takes about 1–3 minutes. HA should be similar. The point is that HA does it at 3 a.m. without you.
- Discuss why a two-host cluster struggles to decide who is dead.

LAB 16 Back up, delete, restore, and prove it boots

EXPECTED RESULTS

- Crash-consistent: like pulling the plug; the disk is in a state a reboot can recover. Application-consistent: the app flushed and paused its own writes first (VSS on Windows, pre-freeze scripts on Linux).
- Image-level backup captures the whole VM (OS, apps, config). In-guest backup captures files the agent can see.
- Concentration risk: one host or datastore failure can take many VMs at once, so backups must live somewhere else.

INSTRUCTOR NOTES

- Emphasize the delete step. A backup is not proven until a restore has booted.
- Fallback for API-restricted hosts: vmrun or ovftool export of a powered-off VM, then import.

LAB 17 Provision a fleet from a definition file

EXPECTED RESULTS

- Idempotence means running the definition twice produces the same result as running it once.
- Infrastructure as code: the file, not the clicks, is the source of truth. It can be reviewed, versioned, and re-run.
- Pets are unique and repaired by hand. Cattle are identical and rebuilt from the definition when they break.

INSTRUCTOR NOTES

- Learners new to PowerShell spend most of the time on syntax. Provide the finished script and have them modify it.

- Optional stretch: the same fleet in Terraform with the vSphere provider in a Hands-on Lab.

LAB 18 Map every concept onto vSphere

EXPECTED RESULTS

- Concept map. CPU ready → %RDY. Ballooning → vmmemctl (MCTL). Page sharing → TPS. Thin disk → thin VMDK. Datastore → VMFS/NFS datastore. Virtual switch → vSphere Standard Switch. Distributed switch → vDS. Live migration → vMotion. Storage migration → Storage vMotion. HA → vSphere HA. Load balancing → DRS. CPU masking → EVC. Template → VM template / Content Library. Resource controls → shares, limits, reservations, resource pools.
- vCenter adds centralized inventory, clustering (HA, DRS), vMotion, distributed switches, templates across hosts, roles, alarms, and a full API.
- The free license has no vCenter integration and a read-only API.

INSTRUCTOR NOTES

- Licensing and editions change frequently. Have learners check the current vSphere edition comparison rather than memorizing it.

LAB 19 Hyper-V VMs, a switch, and replica to a second host

EXPECTED RESULTS

- Hyper-V uses a parent (root) partition that runs Windows and the management stack, and child partitions for VMs. The hypervisor sits beneath both.
- Kerberos authentication needs both hosts in a domain. Certificate authentication works in a workgroup.
- Replica is asynchronous (every 30 s to 15 min): it is disaster recovery with some data loss, not HA.

INSTRUCTOR NOTES

- The certificate steps are fiddly. Consider having learners build one domain controller instead if your class has 32 GB laptops.
- Explain that Hyper-V on a laptop host conflicts with Workstation's nested features; that is why both hosts are VMs here.

LAB 20 A two-node Proxmox cluster with live migration

EXPECTED RESULTS

- KVM: kernel module that turns Linux into a hypervisor using VT-x/AMD-V. QEMU: user-space process that emulates the devices and runs each VM. libvirt/Proxmox: management layers that define, start, and migrate VMs.
- Each running VM is one QEMU process on the host, which is why ordinary Linux tools (ps, top, kill) apply to it.

INSTRUCTOR NOTES

- Proxmox is popular in home labs. Encourage learners to keep this cluster as a free, API-unrestricted platform for later practice.

LAB 21 Deliver a virtual desktop through a broker

EXPECTED RESULTS

- Persistent desktops keep user changes (good for developers, costly to manage). Non-persistent desktops reset on logout (good for call centers and kiosks, simpler to patch).
- A broker authenticates users, picks a desktop, and connects them. Protocols (RDP, Blast, PCoIP, ICA) carry the display.
- Login storms happen when many desktops boot or log in at once, saturating storage IOPS first.

INSTRUCTOR NOTES

- Commercial VDI products (Horizon, Citrix, AVD) are too heavy for a laptop. Guacamole teaches the same broker idea for free.

LAB 22 First containers, measured against a VM

EXPECTED RESULTS

- A container virtualizes the OS: it is an isolated process group on a shared kernel. A VM virtualizes hardware and runs its own kernel.
- Namespaces control what a process can see (PIDs, network, mounts). cgroups control how much it can use (CPU, memory, I/O).
- Containers give up kernel isolation: a kernel exploit in one container can affect the host and all containers on it.

INSTRUCTOR NOTES

- Ask learners to predict the timing ratio before measuring.

LAB 23 Build, persist, and publish an image

EXPECTED RESULTS

- Each Dockerfile instruction creates a layer; unchanged layers are reused from cache, so only the COPY layer rebuilds for 1.1.
- The container's writable layer is deleted with the container. Volumes live outside it.
- A registry stores images by name and tag so any host can pull the same bytes.

INSTRUCTOR NOTES

- Common mistake: editing files inside a running container and expecting them in the image.

LAB 24 A small cluster that survives a node failure

EXPECTED RESULTS

- A deleted pod is replaced in seconds because the deployment controller sees fewer replicas than declared.
- After node loss, Kubernetes waits for the node to be NotReady and then for the default 300-second toleration before evicting and rescheduling pods.
- Pods group containers; deployments keep N pods running and handle rollouts; services give pods a stable address.

INSTRUCTOR NOTES

- The five-minute wait surprises learners. Use it to discuss why fast failover can be dangerous (split-brain again).

LAB 25 Trace one workload through every layer

EXPECTED RESULTS

- kubectl sees pods and nodes as healthy because the fault is below the node: the VM is being starved of CPU by its host.
- This is why the cloud runs Kubernetes on VMs: the VM boundary gives stronger isolation between tenants, and the host layer still needs its own monitoring.

INSTRUCTOR NOTES

- Encourage learners to keep the stack diagram. It becomes the layered map in Lab 28.

LAB 26 A cloud VM, a managed container, and the map back

EXPECTED RESULTS

- Most cloud instances report KVM or a KVM-derived hypervisor (e.g. Nitro on AWS).
- You manage the guest OS, data, identity, and network rules. The provider manages the hypervisor, hardware, and physical network.
- Serverless and managed containers hide the VM entirely; you are billed per request or per second instead of per instance-hour.

INSTRUCTOR NOTES

- Warn learners about cleanup twice. Forgotten resources are the most common real-world cost.

LAB 27 Harden the management plane and scan an image

EXPECTED RESULTS

- The hypervisor is the highest-value target: whoever controls it controls every VM on it.

- VM escape is rare. The realistic path is stolen admin credentials or an exposed management interface, which is why this lab hardens access first.
- Templates freeze whatever was in them: keys, tokens, history. Clean them before sealing (Lab 12) and scan images before shipping.
- Containers share the host kernel, so an unpatched base image or kernel is a risk to every container on the host.

INSTRUCTOR NOTES

- Have learners take the esxi1 snapshot. Someone always firewalls themselves out.

LAB 28 Diagnose a multi-layer fault to root cause

EXPECTED RESULTS

- (a) Storage path lost: VM freezes; vmkernel.log shows NFS server not responding; affects every VM on esxi1 using nas-nfs, none on esxi2.
- (b) Datastore full: VMs pause with out-of-space questions; every VM on nas-nfs on both hosts.
- (c) Storage latency: everything on nas-nfs slow; esxtop LAT/rd high; nothing in guest logs points at the cause.
- (d) Snapshot growth: only tiny01 slower; datastore browser shows a growing -000001.vmdk.
- (e) CPU limit: only tiny01; esxtop %MLMTD (time held back by the limit) high while host is idle.

INSTRUCTOR NOTES

- If running in class, have partners inject faults for each other instead of the random script.
- Grade on the evidence trail, not on speed.

LAB 29 Attribute four staged slowdowns

EXPECTED RESULTS

- A: victim bogo ops/s drops about half; %RDY per vCPU 30–60%.
- B: host state soft/hard; victim MCTLSZ or SWCUR above 0; everything sluggish.
- C: noisy neighbor has thousands of CMDs/s; victim LAT/rd rises from under 1 ms to 10+ ms. Storage is the most common cause of 'everything is slow'.
- D: vmnic near line rate in esxtop n; only traffic crossing that uplink is affected.

INSTRUCTOR NOTES

- Nested latency numbers are distorted by the laptop SSD. Teach relative change, not absolute thresholds.

LAB 30 Recover from five staged failures

EXPECTED RESULTS

- Power-on failures are VM config or host resources. Boot failures are virtual hardware (disk, boot order, firmware) or the guest OS. The console tells them apart: no boot device = virtual hardware; kernel panic = guest.
- Recovery ladder: repair when the cause is known and quick; restore when data is damaged or repair is slow; rebuild from template when the VM is cattle and data lives elsewhere.
- The host-failure answer should be 'register on esxi2', because the files are intact on shared storage. Restore would lose data since the last backup.

INSTRUCTOR NOTES

- Run failures in sequence with a short debrief after each.

LAB 31 Size the host and reclaim the waste

EXPECTED RESULTS

- Example: 4 pCPUs, 6 vCPUs = 1.5:1 (fine). 4.5 GB configured on a 5 GB host with ~1 GB ESXi overhead = memory is the limiting factor.
- tiny01 and tiny03 are oversized: 2 vCPUs and 1.5 GB with under 5% CPU and under 150 MB active. Right-size to 1 vCPU and 768 MB.
- N+1 on two 5 GB hosts: all VMs must fit in about 4 GB usable on one host. If they do not, you do not have real HA capacity.

INSTRUCTOR NOTES

- Memory is almost always the limit in labs and in production; CPU rarely is.

LAB 32 Assess yourself and write a 90-day plan

EXPECTED RESULTS

- A good plan is specific ('rebuild the Lab 15 recovery from memory by day 20'), not general ('learn HA').
- The concepts that last: abstraction, the four resources, contention, shared storage, layered troubleshooting. Platform names change; these do not.

INSTRUCTOR NOTES

- If running a cohort, have learners present their plans in pairs and hold each other to the 30-day check-in.